UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II

Appunti di
Linguaggi di Programmazione I

Anno 2021

Valentino Bocchetti

Contents

1 Introduzione al corso e informazioni **10**

1.1 Esame 10

2 Suddivisione del corso **10**

2.1 Parte I 10

2.2 Parte II (modello object-oriented, Java) 10

2.3 Parte III (linguaggi funzionali, ML) 11

2.4 Parte IV (linguaggi logici, Prolog) 11

3 Lezione del 09-03 **12**

3.1 Breve storia 12

3.2 Definizione di base 13

3.3 Linguaggi completi 13

3.4 Macchine astratte 14

3.5 Traduttori del linguaggio 16

3.6 SRT 18

3.7 Compilazione ed esecuzione 18

3.8 Proprietà dei linguaggi 20

3.9 Criteri di scelta dei linguaggi 20

4 Lezione del 12-03 **20**

4.1 Paradigmi computazionali 20

4.1.1 Paradigma imperativo + esempio (pseudo-linguaggio + c): 21

4.1.2 Paradigma funzionale + esempio (pseudo-linguaggio + lisp): 22

4.1.3 Paradigma logico + esempio (prolog): . . . 22
4.1.4 Imperativo vs funzionale . . . 23
4.1.5 Orientato ad oggetti . . . 23
4.1.6 Parallelo . . . 24
4.2 Modello Imperativo nel dettaglio . . . 24
4.3 Esempi di semantica di assegnamenti in C . . . 26
4.3.1 Assegnazione . . . 26
4.3.2 Operatore & . . . 26
4.3.3 Operatore & a sinistra . . . 26
4.3.4 Puntatori (a destra) . . . 26
4.3.5 Puntatori (a sinistra) . . . 27
4.3.6 vettori e puntatori . . . 27

5 Lezione del 16-03 **29**
5.1 Data Object e legami . . . 29
5.1.1 Variazione dei legami . . . 29
5.1.2 Legame di tipo . . . 31
5.1.3 Type checking . . . 32
5.1.4 Il linguaggio perfetto . . . 32
5.1.5 Strategie . . . 33
5.2 Blocchi di istruzioni . . . 33
5.2.1 Ambito di validità di legami . . . 34
5.2.2 Legami di nome . . . 34
5.2.3 Mascheramento . . . 35

5.2.4 Legami di locazione 35

5.2.5 Stack di attivazione 36

6 Lezione del 19-03 **37**

6.1 Procedure come astrazioni 37

6.2 Record di attivazione 38

6.2.1 Ambiente locale 38

6.2.2 Propagazione dei Data OBJECT 38

6.3 Parametrizzazione di procedure 41

6.3.1 Parametri IN 42

6.3.2 Parametri OUT 43

6.3.3 Parametri IN-OUT 43

6.4 Aliasing 43

6.5 Procedure come parametri di procedure 43

7 Lezione del 23-03 **44**

7.1 Memo 44

7.2 Funzioni 44

8 Lezione del 26-03 **44**

8.1 Esempi sul record di attivazione nei linguaggi 44

8.1.1 Esempio interferenza 44

8.1.2 Esempio copia 45

8.2 Macro 45

8.3 Implemantazione efficiente dello scoping statico 46

8.4 Annidamento 46

8.5 Rappresentazione variabili non locali 47

8.6 Mantenimento del vettore degli ambienti non locali 48

8.7 Proprietà di questa implementazione 48

8.8 Paradigma OO 49

8.9 Encapsulation 49

8.10 C++ vs Java 49

8.11 UML 49

8.12 Essenza di un oggetto 49

8.13 Classe 50

8.14 Polimorfismo 50

9 Lezione del 30-03 **50**

9.1 Classi astratte 50

9.2 Visibilità delle componenti di una classe 50

9.3 Ereditarietà 51

9.3.1 Generalizzazione 51

9.3.2 Specializzazione 51

9.4 Associazioni e molteplicità 51

9.4.1 Associazioni complesse 51

9.4.2 Ruoli nelle associazioni 51

9.4.3 Associazioni riflessive 52

9.5 Introduzione a Java 52

9.5.1 Errori comuni 52

9.5.2 Compilazione ed Esecuzione 53

9.5.3 Sicurezza 53

9.5.4 Costruttori 54

10 Lezione del 09-04 **54**

10.1 Organizzazione della memoria nei linguaggi di programmazione 55

10.2 Ambiente non locale nel paradigma a oggetti 55

10.3 Java 55

10.3.1 Commenti 55

10.3.2 Blocchi 56

10.3.3 Identificatori 56

10.3.4 Tipi primitivi 56

10.3.5 Tipo String 56

10.3.6 Tipi Reference 56

10.3.7 Costruzione degli oggetti 56

10.3.8 Parametri 57

10.3.9 Il riferimento this 57

10.3.10 Convenzioni sul codice 57

10.4 Conversioni 57

10.4.1 Type equivalence 58

10.4.2 Compatibilità di tipi 58

10.5 Variabili 58

10.6 Inizializzazione 59

11 Lezione del 13-04 **59**

11.1 Operatori in Java 59

11.1.1 Booleani 59
11.1.2 Di bit 59
11.1.3 Right e Left Shift 60
11.1.4 Concatenazione di stringhe 60
11.2 Casting di primitivi 60
11.2.1 Conversioni legali 61
11.2.2 Conversioni esplicite 61
11.2.3 Promozione aritmetica 61
11.3 Enunciati Branch 62
11.3.1 If & else 62
11.3.2 Switch 62
11.4 Enunciati loop 62
11.4.1 For 62
11.4.2 While 63
11.4.3 `do/while` 63
11.5 Controlli speciali 63

12 Lezione del 16-04 **63**

12.1 Array 63
12.1.1 Creazione 63
12.1.2 Inizializzazione 64
12.1.3 Multidimensioni 65
12.1.4 Estremi 65
12.1.5 Assegnazione 65

12.1.6 Ridimensionamento . . . 65
12.1.7 Copia . . . 65
12.1.8 Aiutare il GC . . . 65
12.2 Ereditarietà . . . 65
12.2.1 Specializzazione . . . 65
12.3 Polimorfismo . . . 66
12.4 Collezioni eterogenee . . . 66
12.4.1 Collezioni omogenee di oggetti dela stessa classe . . . 66
12.4.2 Collezioni eterogenee di oggetti di classi diverse: . . . 66
12.5 Argomenti polimorfici . . . 66
12.6 Operatore `instanceof` . . . 66
12.7 Casting . . . 67
12.8 Composizioni . . . 67
12.9 Aggregazioni . . . 67
12.10 Associazioni . . . 68
12.11 Overloading e Overriding . . . 68
12.11.1 La parola chiave `super` . . . 68
12.12 Costruzione ed inizializzazione di oggetti . . . 68
12.13 La classe `Object` . . . 69
12.13.1 Il metodo `equals` . . . 69
12.13.2 Il metodo `toString` . . . 69
12.14 Le classi `wrapper` . . . 70
12.14.1 Uso . . . 70
12.15 Molteplicità . . . 70

13 Lezione del 26-04 **70**

13.1 Polimorfismo per inclusione (basato su upcast e downcast) 70

13.2 Modificatori di accesso 70

13.2.1 Generalità 71

13.2.2 Altri modificatori 72

13.2.3 Sommario 76

13.2.4 Singoletto 76

1 Introduzione al corso e informazioni

`Email` → pab@unina.it

`Ricevimento` → da richiedere in precedenza con una mail

1.1 Esame

- Uno scritto java + altre parti del corso;
- In caso di dubbi c'è un orale;

Lo scritto può essere mantenuto per la durata di un anno.

2 Suddivisione del corso

2.1 Parte I

- Storia e concetti di base;
- Primi cenni ai paradigmi dei linguaggi di programmazione;
- Il modello imperativo;

2.2 Parte II (modello object-oriented, Java)

- Studio dei costrutti più avanzati
  - qualificatori di classi;
  - metodi e attributi;
  - classi astratte;
  - interfacce;
  - template;
  - classi interne;
  - gestione degli errori;
  - eccezioni;

2.3 Parte III (linguaggi funzionali, ML)

- ML e costrutti avanzati;
- funzioni di ordine superiore;
- polimorfismo;
- tipi di dato astratti e interfacce;
- template;
- gestione degli errori;
- eccezioni;
- type inference;

2.4 Parte IV (linguaggi logici, Prolog)

- Costrutti fondamentali
  - termini;
  - predicati;
  - regole;
- Tecniche di programmazione avanzate
  - invertibilità;
  - nondeterminismo;

3 Lezione del 09-03

3.1 Breve storia

Il primo linguaggio introdotto è il FORTRAN (introdotto con lo scopo di calcoli scientifici). Nasce la concezione di variabile

Anno	Linguaggio
1954	FORTRAN (FORmula TRANslation)
1960	COBOL
	ALGOL (60 e 68)
	`PL/1`
	Simula 67
	PASCAL
	LISP
	APL
	BASIC
`1970/1980`	PROLOG
	SMALLTALK
	C
	`MODULA/2`
	ADA

- **Cobol** → orientato ai database (introduce i record);
- **Agol60** → indipendenza dalla macchina e definizione mediante grammatica (supporto dell'iterazione e ricorsione, introduzione delle strutture a blocco);
- **Lisp** → primo vero linguaggio di manipolazione simbolica, linguaggio funzionale (non esiste lo statement di assegnazione);
- **Prolog** → primo (e principale) linguaggio di programmazione logica, porta l'invertibilità e una programmazione in stile nondeterministico (essenzialmente non tipato; queste vengono ottenute mediante metaprogrammazione);
- **Simula 67** → classe come incapsulamento di dati e procedure, istanze delle classi (oggetti);
- `PL/1` abilità ad eseguire procedure specificate quando si verifica una condizione eccezionale; (multitasking);
- **Pascal** → programmazione strutturata, tipi di dato definiti da utente (ricchezza di strutture dati); nessuna encapsulation.

3.2 Definizione di base

Linguaggio di programmazione

È un linguaggio che è usato per esprimere (mediante un programma) un processo con il quale un processore può risolvere un problema.

Processore

È la macchina che eseguirà il processo descritto dal programma (da non intendere come un singolo oggetto, ma coome una *architettura di elaborazione*).

Programma

È l'espressione codificata di un processo.

3.3 Linguaggi completi

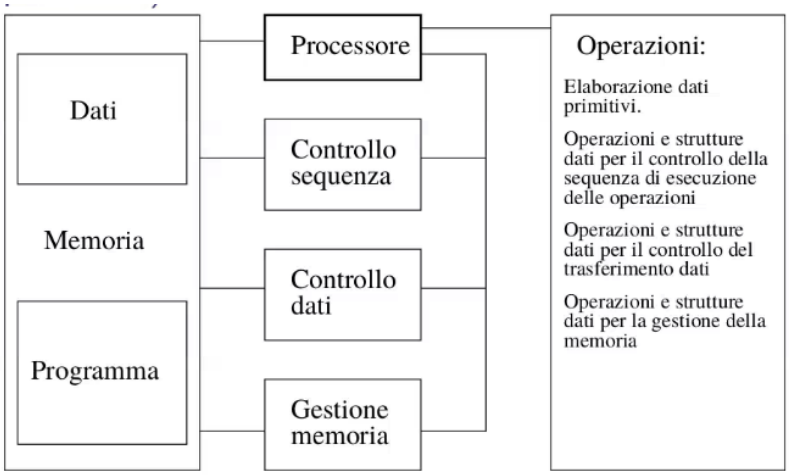
Sono linguaggi in grado di programmare qualunque funzione calcolabile (*general purpose*). Tecnicamente devono poter simulare qualunque *macchina di Turing.*

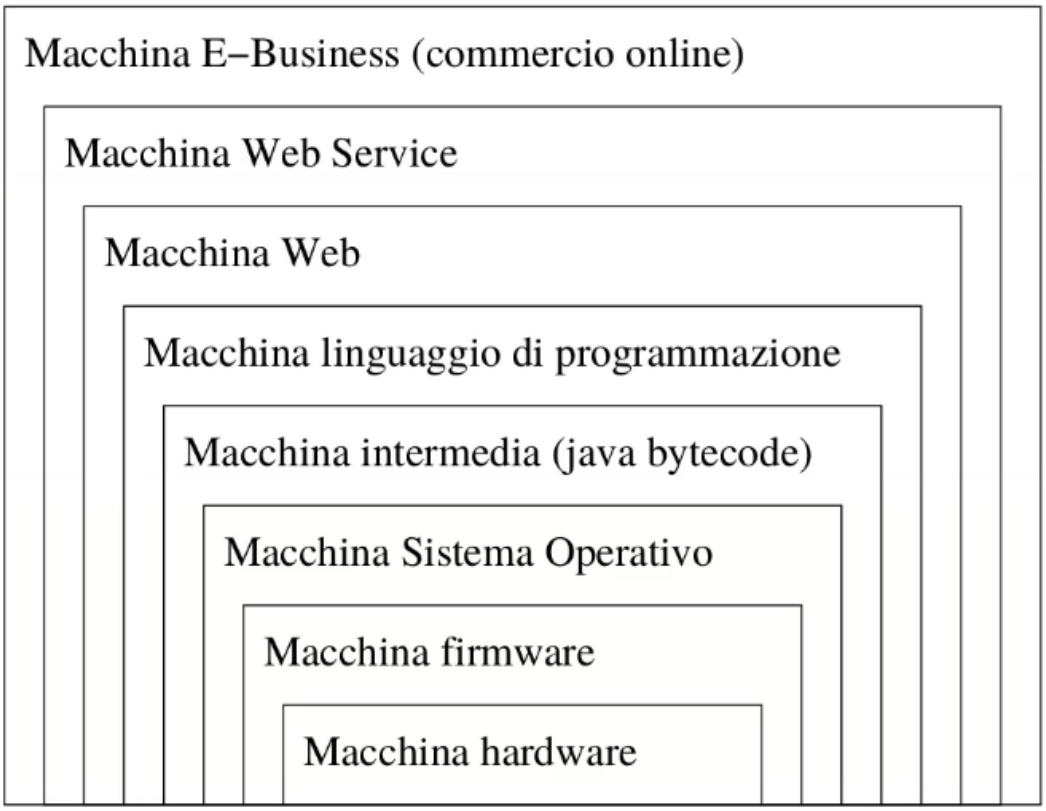
Sono completi solo quelli che riescono ad esprimere anche programmi di cui non è decidibile la terminazione. Per dimostrare la completezza di un linguaggio lo si usa per simulare arbitrarie macchine di Turing.

Ad esempio non sono linguaggi completi **SQL** (perchè la terminazione dei programmi è sempre decidibile; è spesso immerso in linguaggio completo) e **HTML**.

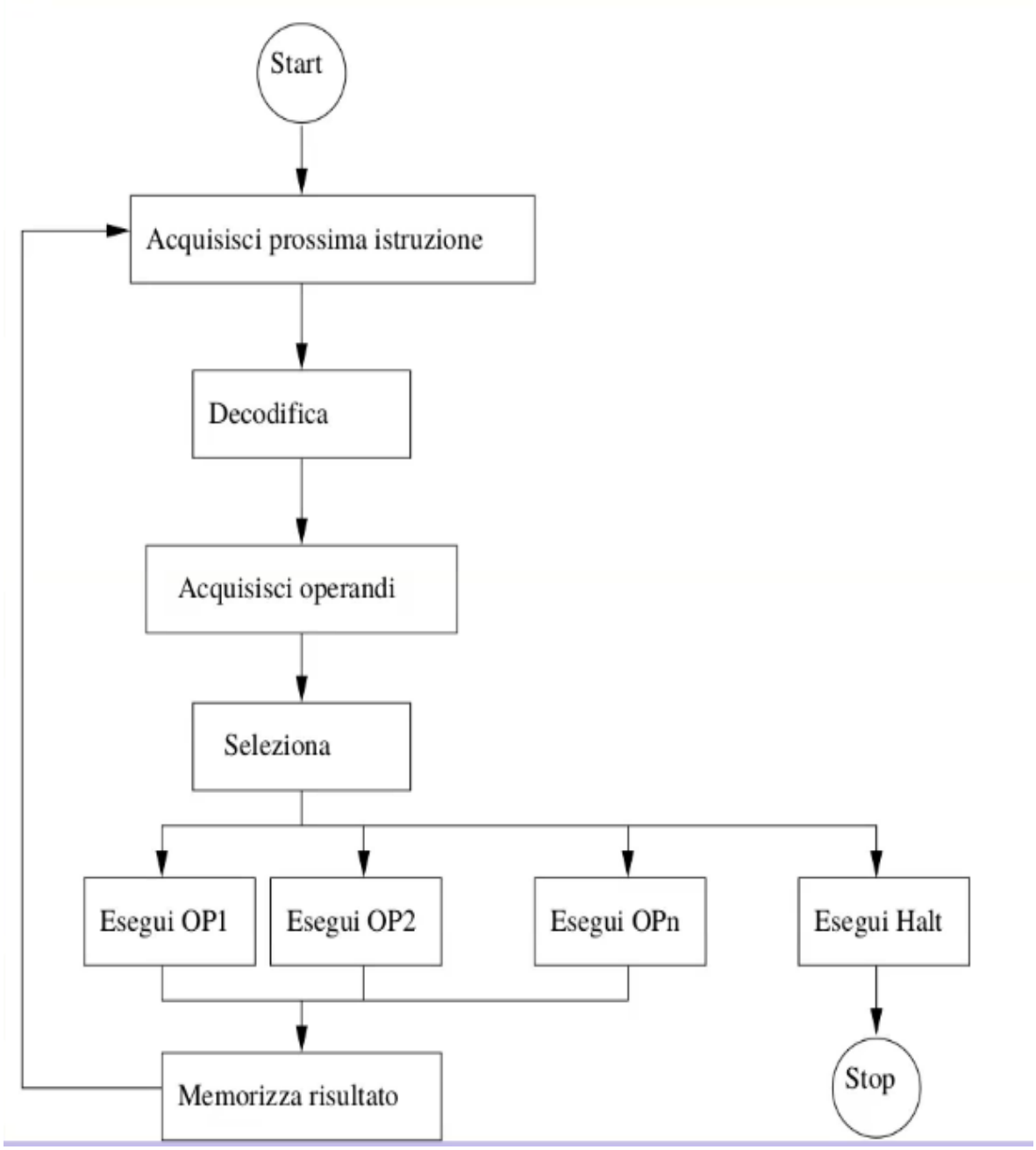
3.4 Macchine astratte

Dato un linguaggio di programmazione L, una macchina astratta per L (in simboli `M_L`) è un qualsiasi insieme di strutture dati e algoritimi che permettano di memorizzare ed eseguire programmi scritti in L. La struttura di una macchina astratta è (essenzialmente memoria e processore):





Processore della macchina astratta



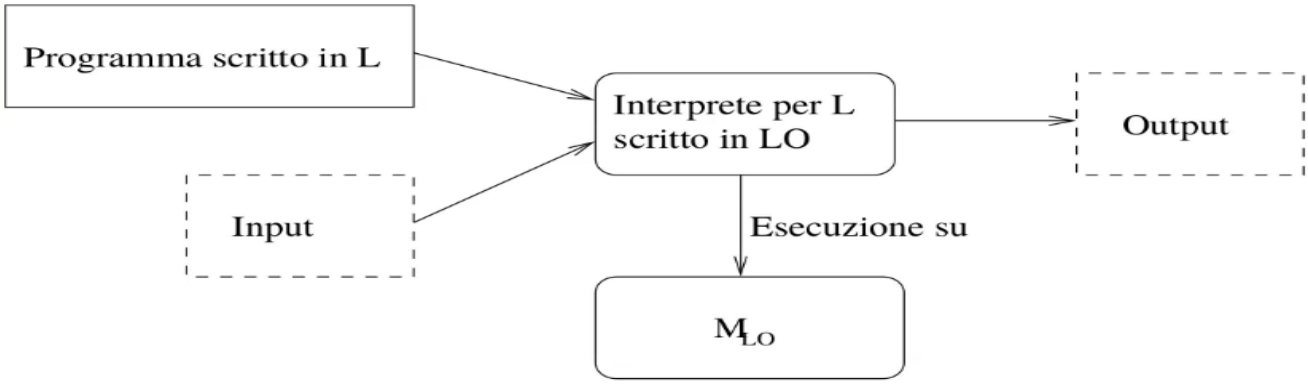
Nel tempo si è pensato a vari modi di implementarlo:

- Hardware (es per *Lisp* e *Prolog*), ma abbandonato rapidamente;
- Firmware → soluzione più flessibile e semplice (ed economico);
- Software → come nel caso della JVM;

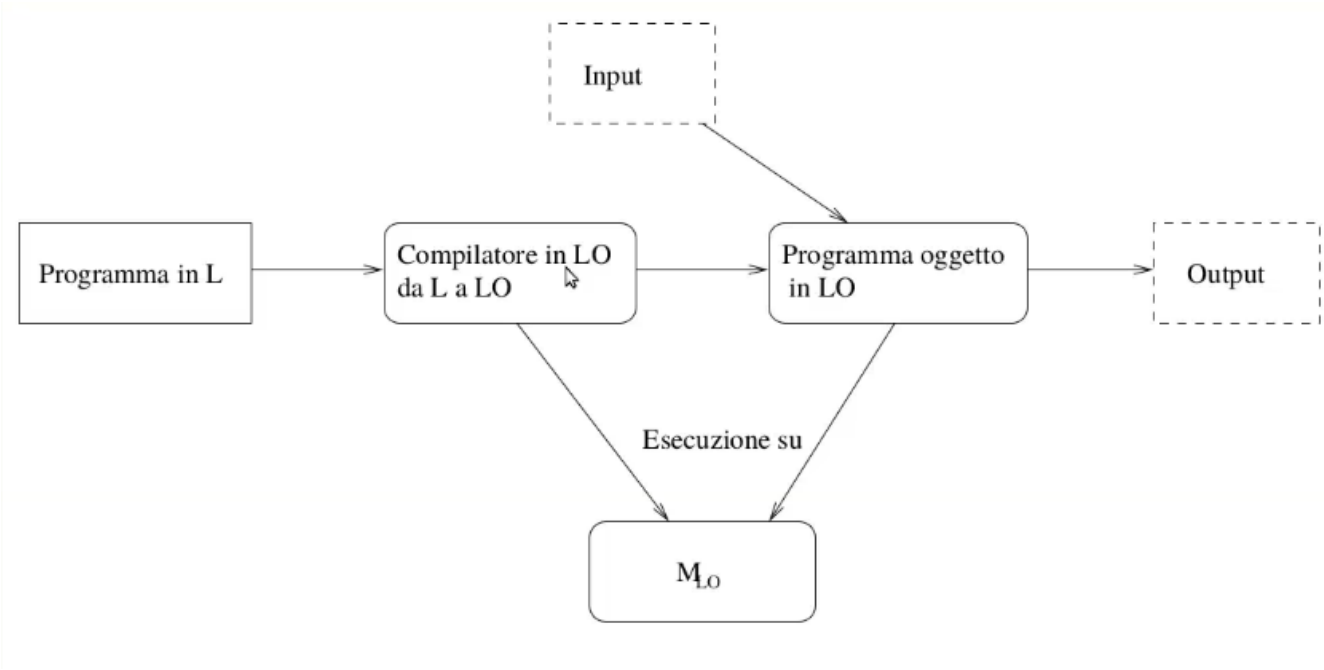
3.5 Traduttori del linguaggio

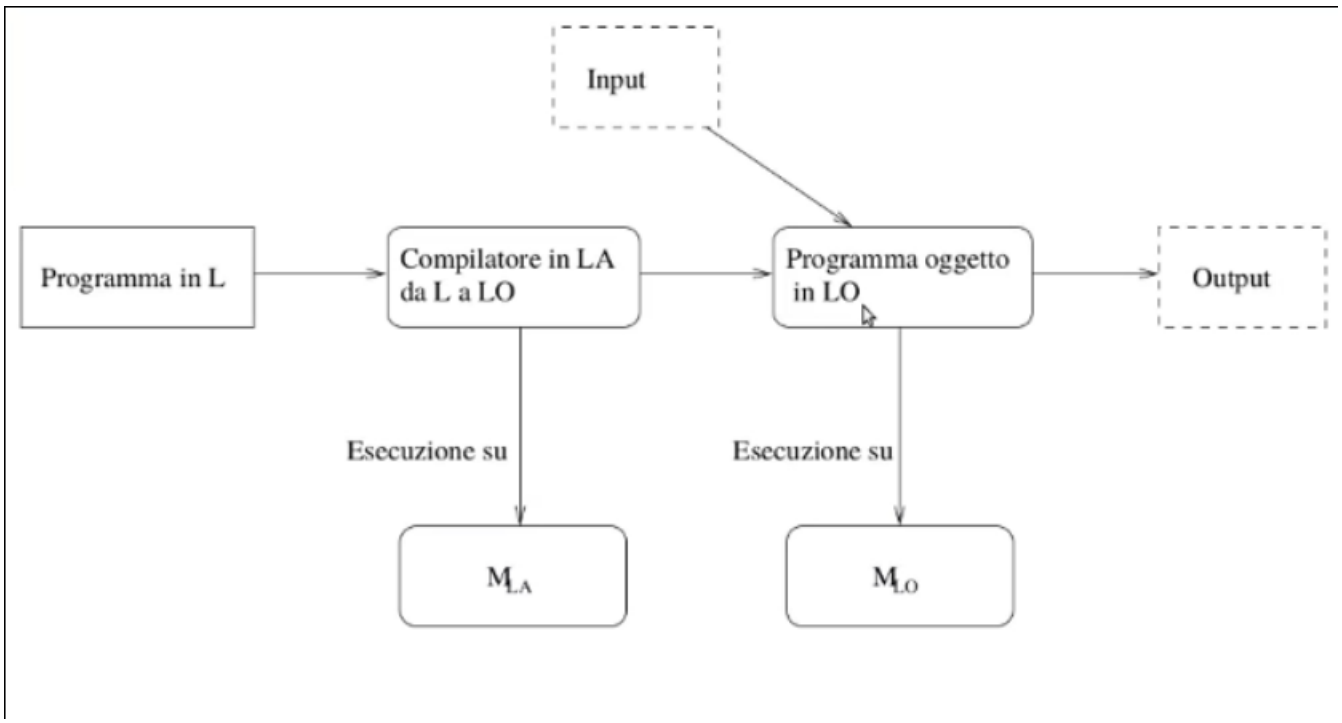
- **Interpreti** → traducono ed eseguono un costrutto alla volta
  - PRO : debug;
- **Compilatori** → prima traducono l'intero programma, che poi può essere eseguito
  - PRO : velocità di esecuzione finale;
  - PRO : più controlli e in anticipo.

Interpretazione pura



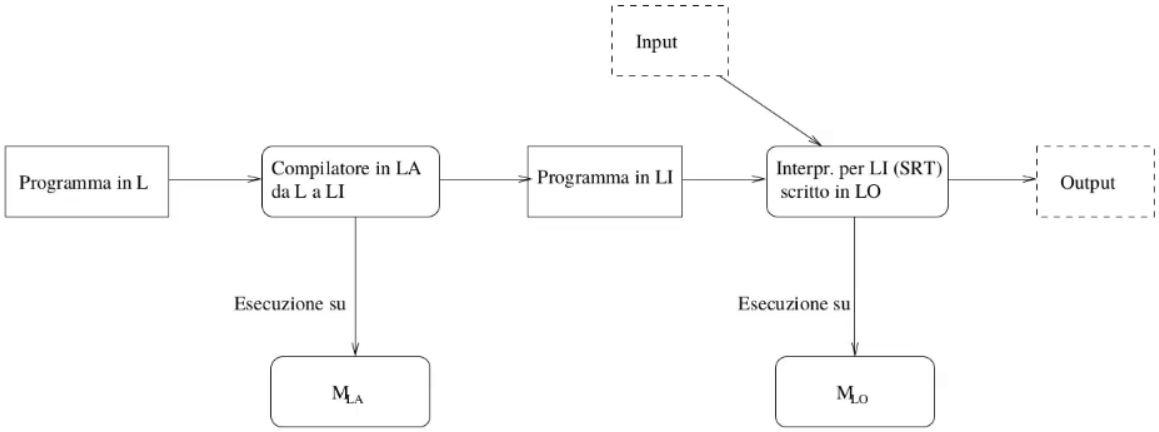
Compilazione pura (caso semplice e generale)





Nella compilazione pura può accadere che L ed LO siano uguali (il vantaggio è il riutilizzo del compilatore $\rightarrow$ **custom compilation**)

Compilazione per macchina intermedia



($SRT \rightarrow$ supporto al run time)

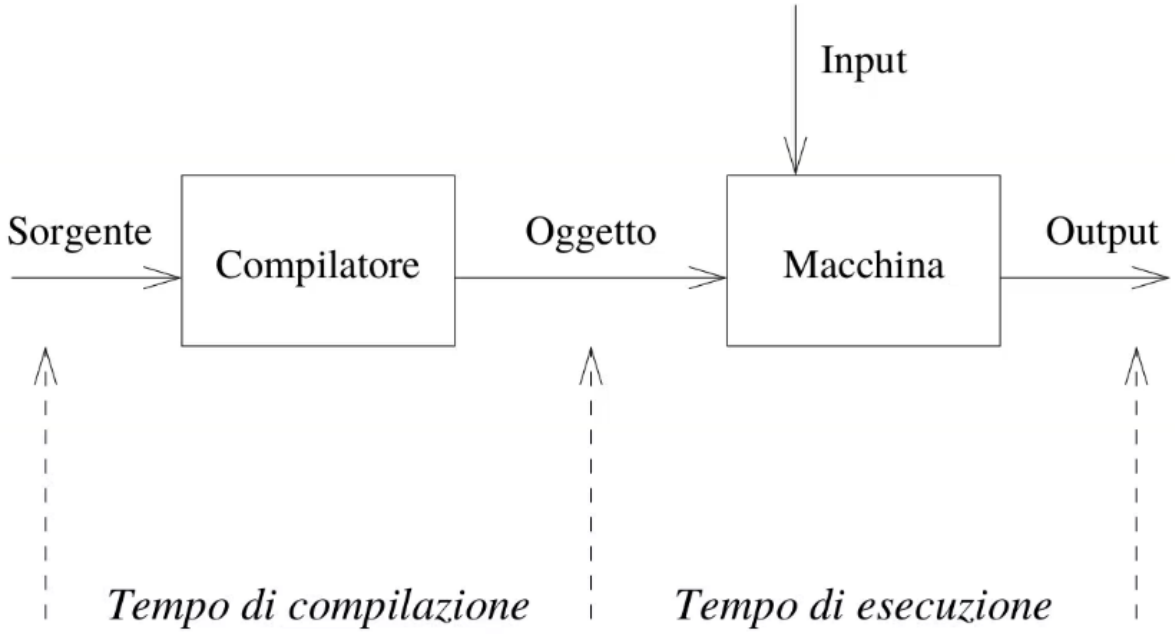
Torna utile nel caso di linguaggi molto diversi dalla macchina astratta sottostante. Invece di creare hardware apposito creiamo un interprete che lo sostituisca

3.6 SRT

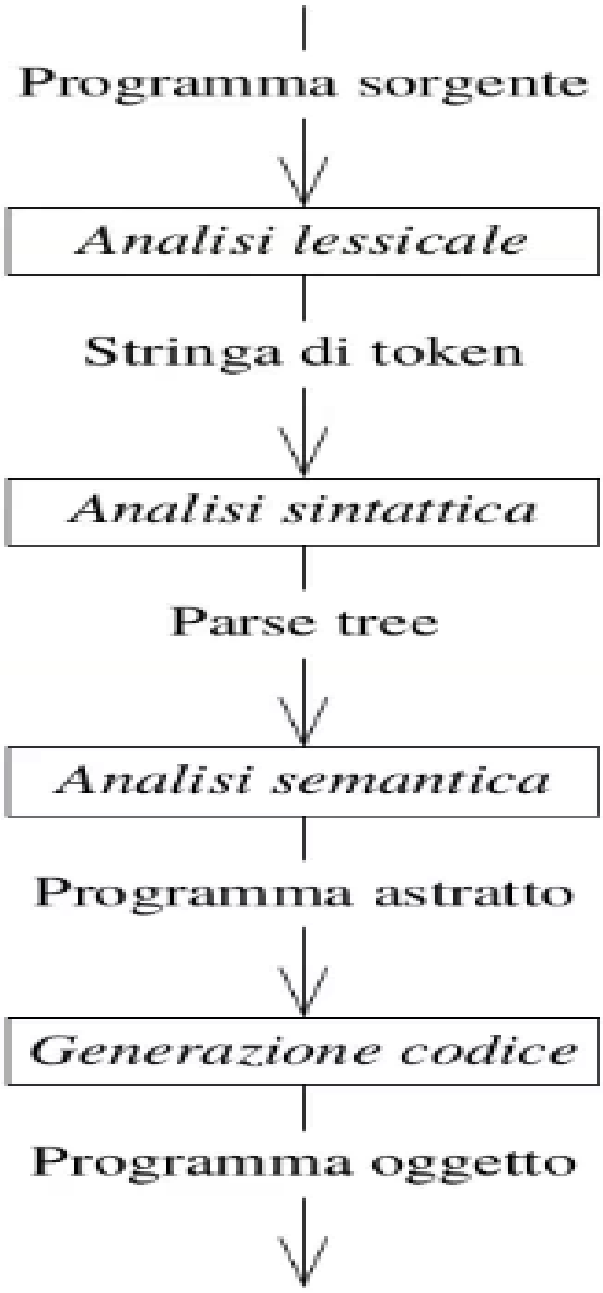
- Funzionalità aggiuntive (rispetto alla macchina astratta sottostante);
- Funzioni di basso livello / interfacce con il S.O. (es `I/O`);
- Gestione della memoria;
- Garbage collection;
- Gestione dell'heap;
- Gestione dello stack;

Non è necessariamente una macchina astratta radicalmente diversa, ma a volte solo un pacchetto di funzioni o librerie aggiunte automaticamente al codice oggetto.

3.7 Compilazione ed esecuzione



Fasi della compilazione



(Analisi lessicale e sintattica sono stati un problema → venivano fatte in maniera artigianale)

3.8 Proprietà dei linguaggi

- **Semplicità** → (coincisione) VS (leggibilità)
  - **Semantica** → minimo numero di concetti e strutture;
  - **Sintattica** → unica rappresentabilità di ogni concetto;
- **Astrazione** → rappresentare solo attributi essenziali
  - **Dati** → nascondere i dettagli di oggetti;
  - **Procedure** → facilitare la modularità del progetto;
- **Espressività** → (facilità di rappresentazione di oggetti) VS (semplicità);
- **Ortogonalità** → meno eccezioni alla regole del linguaggio;
- **Portabilità.**

3.9 Criteri di scelta dei linguaggi

- Disponibilità dei traduttori;
- Maggiore conoscenza da parte del programmatore;
- Esistenza di standard di portabilità (es ADA);
- Comodità dell'ambiente di programmazione;
- Sintassi aderente al problema;
- Semantica aderente alla architettura fisica non importante (non è più un criterio stringente).

4 Lezione del 12-03

4.1 Paradigmi computazionali

Il paradigma di appartenenza può influenzare radicalmente il modo in cui si risolve il problema, ma non è l'unica discriminante. Infatti sono discriminanti anche:

- Il sistema di tipi supportato;
- Eventuale supporto alle eccezioni;
- Modello di concorrenza e sicronizzazione.

Sono presenti tre tipi di paradigmi computazionali:

- Imperativo;
- Funzionale;
- Logico.

es. Un programma specifica sequenze di modifiche da apportare allo stato della macchina (memoria)

Vogliamo scrivere in tutti e tre i linguaggi la funzione membro (X, L), che decida se l'elemento X appartiene alla lista L

```
membro(2, [1,2,3]) = true; membro(2, [1,2,3]) = false;
```

A questo scopo supponiamo siano già esistenti le funzioni:

- vuota(L), che restituisce true se L è vuota, false altrimenti;
- testa(L), che restituisce il primo elemento della lista L;
- coda(L), che restituisce una sottolista ottenuta rimuovendo il primo elemento di L

4.1.1 Paradigma imperativo + esempio (pseudo-linguaggio + c):

```
procedure membro (X,L)
    local L1 = L
    while not vuota(L1) and not X=testa(L1)
do L1 = coda(L1)
return not vuota(L1)


bool member(X, L){
    List L1 = L;
    while ( ! empty(L1) && ! X=testa(L1))
L1 = coda(L1);
    return (! vuota(L1));
}
```

4.1.2 Paradigma funzionale + esempio (pseudo-linguaggio + lisp):

Il programma e le sue componenti sono funzioni. Esecuzione come valutazioni di funzioni

```
function member(X, L)
if vuota (L) then false
else if X == testa(L) then true
else member(X, coda (L))
```

```
(defun membro (x l)
    (cond ((null l) nil)
((equal x (first)) T)
(T (membro x (rest l)))))
```

Notiamo che anche il c può prestarsi alla versione funzionale

```
bool member(X, L){
    return (vuota (L)) false :
(X == testa(L))? true :
member(X, coda(L))}
```

4.1.3 Paradigma logico + esempio (prolog):

Programmma come descrizione logica di un problema. Esecuzione analoga a processi di dimostrazione di teoremi

```
membro(X, [X|L]) .
membro(X, [Y|L]) : - membro(X, L).
```

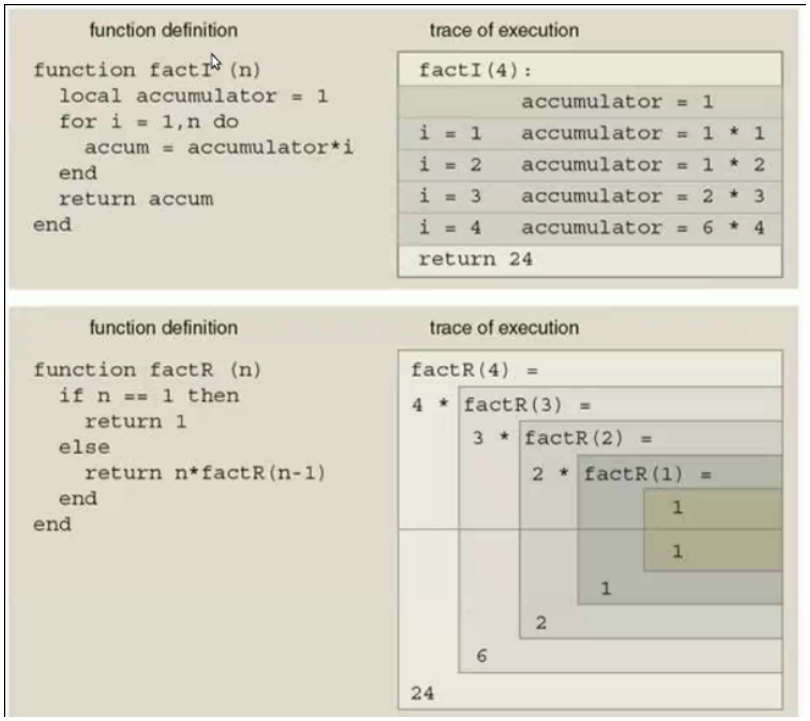
Notiamo come i parametri formali di member possono essere pattern (consistono di definizioni di predicati). Esecuzione:

`member(2,[1,2,3]) restituisce yes (true)` `member(0,[1,2,3]) restituisce no (false)`

Presenta diverse proprietà:

- Si comporta come un generatore → query con variabili: member(X,[1,2,3]) restituisce
  - X=1;
  - X=2;
  - X=3;
  - no;
- Propone risposte con variabili: member(1, L) restituisce
  - `L=[1| L_0], L=[Y_0, 1 | L_1]; L = [Y_0, Y_1, 1 | L_2] ...`
- Invertibilità → nessuna distinzione tra input e output → un solo predicato (programma), molte funzioni;

4.1.4 Imperativo vs funzionale



Si noti l'eleminazione degli assegnamenti:

- n rimpiazza i (ricorsione invece di cicli)

4.1.5 Orientato ad oggetti

Programma costituito da ogetti che scambiano messaggi.

4.1.6 Parallelo

Programmi che descrivono entità distribuite che sono eseguite contemporaneamente e in modo asincrono.

4.2 Modello Imperativo nel dettaglio

Consiste in un insieme di "contenitori di dati":

- Ad es. parole (o celle) della memoria centrale;
- Tipicamente rappresentate dal loro indirizzo

Concettualmente la memoria quindi è:

- Una funzione da uno spazio di locazioni ad uno spazio di valori;
- mem(loc) = "valore contenuto in loc"

La sintassi è del tipo

`<assegnazione>` ::= `<name>` `<assigment-operator>` `<expression>`

dove:

- `<name>` rappresenta la locazione dove viene posto il risultato;
- `<expression>` vengono specificati una computazione e i riferimenti ai valori necessari alla computazione

In Pascal:

```
a := b + c;
```

Il valore di `<expression>` va memorizzato nell'indirizzo rappresentato da `<name>`. Il valore di `<expression>` dipenderà dai valori contenuti negli indirizzi degli argomenti di `<expression>` rappresentati dai nomi di questi ottenuto seguendo le prescrizione del codice associato al suo nome

Questo tipo di modello è il più vicino agli elaboratori "standard". I programmi sono descrizioni di sequenze di modifiche della "memoria" del calcolatore (come nel caso del modo di pilotare i display, salvare su memorie periferiche). Ogni esecuzione consiste di 4 passi (in sostanza un assrgnamento):

- Ottenere indirizzi delle locazioni di operandi e risultato;
- Lettura degli operandi;
- Valutazione del risultato;
- Memorizzazione del risultato

In questo tipo di modello le variabili di un programma costituiscono una memoria, mentre i parametri formali delle `funzioni/procedure` no (sia concettualmente che per come sono realizzati). L'ambiente (environment) comprende:

- Un insieme di nomi di variabili e parametri
  - non indirizi di memoria, piuttosto identificatori, associati al loro valore (direttamente e non)

Nel paradigma imperativo, la funzione env (ambiente) associa gli identificatori a locazioni di memoria, le quali, a loro volta, sono associate (funzione mem) al contenuto di memoria. Il valore di una variabile x è mem(env(x)). In questo tipo di paradigma la funzione env identifica una associazione immutabile (la locazione di memoria associata a un nome non cambia)

Nel paradigma funzionale, non esiste la funzione mem e la funzione env associa direttamente gli identificatori al valore della memoria

ATTENZIONE: env(x) è immutabile finchè esiste x (gli identificatori nascono e muoiono durante l'esecuzione)

In una assegnazione del tipo `x := x +1;` vediamo che il valore di destra va salvato nella variabile di sinistra:

- la x di sinistra indica la locazione associata al nome (cioè env(x));
- la x di destra indica il valore della memoria (cioè mem(env(x)))

4.3 Esempi di semantica di assegnamenti in C

4.3.1 Assegnazione

```
x=y
```

In questo caso avremo:

- sx:(env(x));
- dx: mem(env(y));

4.3.2 Operatore &

```
int *x; y
x = &y;
```

In questo caso avremo:

- sx: env(x);
- dx: env(y);

4.3.3 Operatore & a sinistra

```
&y = x;
```

`NB: ERRORE!` → sarebbe come scrivere env(env(y)) che è malformato (env accetta solo cose che siano identificatori)
Il C riporta errore di sintassi *not an lvalue*. Valori corretti possono essere:

- nome di variabile;
- `*(<pointer-expression>)`;

4.3.4 Puntatori (a destra)

```
y = *x;
```

In questo caso avremo:

- sx: env(x);
- dx: mem(mem(env(y)));

4.3.5 Puntatori (a sinistra)

In C `*x` indica la cella di memoria puntata da x (la cella di memoria il cui indirizzo sta scritto dentro x)

```
*x = y;
```

In questo caso avremo:

- sx: mem(env(x));
- dx: mem(env(y))

4.3.6 vettori e puntatori

```
int v[3];
```

In questo caso env(v) indica la prima cella di memoria del vettore v

y = v[1];

In questo caso avremo:

- sx: env(y);
- dx: mem(env(v) +1);

v[1] = y;

In questo caso avremo:

- sx: env(v) +1;
- dx: mem(env(y));

y = x[1] + v[1]; //puntatore + vettore

In questo caso avremo:

- sx: env(v) +1;

- dx: mem(men(env(x) +1)) + mem(env(v) + 1);

```
x[1] = y
```

In questo caso avremo:

- sx: mem(env(x)) +1

```
y = *x + *v
```

In questo caso avremo:

- dx: mem(mem(env(x))) + mem(env(v))

```
*v = y;
```

In questo caso avremo:

- sx: env(v); (visto come v[0] sarebbe env(v) + 0)

```
y = *(v+2)
```

In questo caso avremo:

- dx: mem(env(v) + 2);

```
y = (*v) + 2
```

In questo caso avremo:

- dx: mem(env(v)) + 2;

```
*(x+2) = ...
```

In questo caso avremo:

- sx: mem(env(x))+2

5 Lezione del 16-03

5.1 Data Object e legami

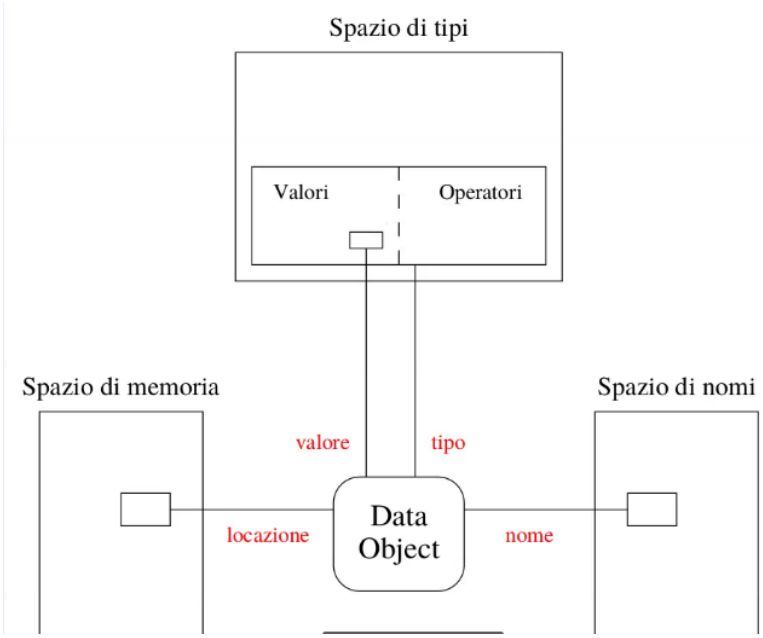
Ogni variabile, parametro, oggetto può essere rappresentato da un Data Object.

È una quadrupla (L, N, V, T) → locazione, nome, valore, tipo.

Legame → determinazioni di una delle componenti

Nel paradigma imperativo:

- Il legame di locazione corrisponde alla funzione *env*;
- il legame di valore di una variabile di nome x corrisponde a *mem(env(x))*.



5.1.1 Variazione dei legami

Possono avvenire:

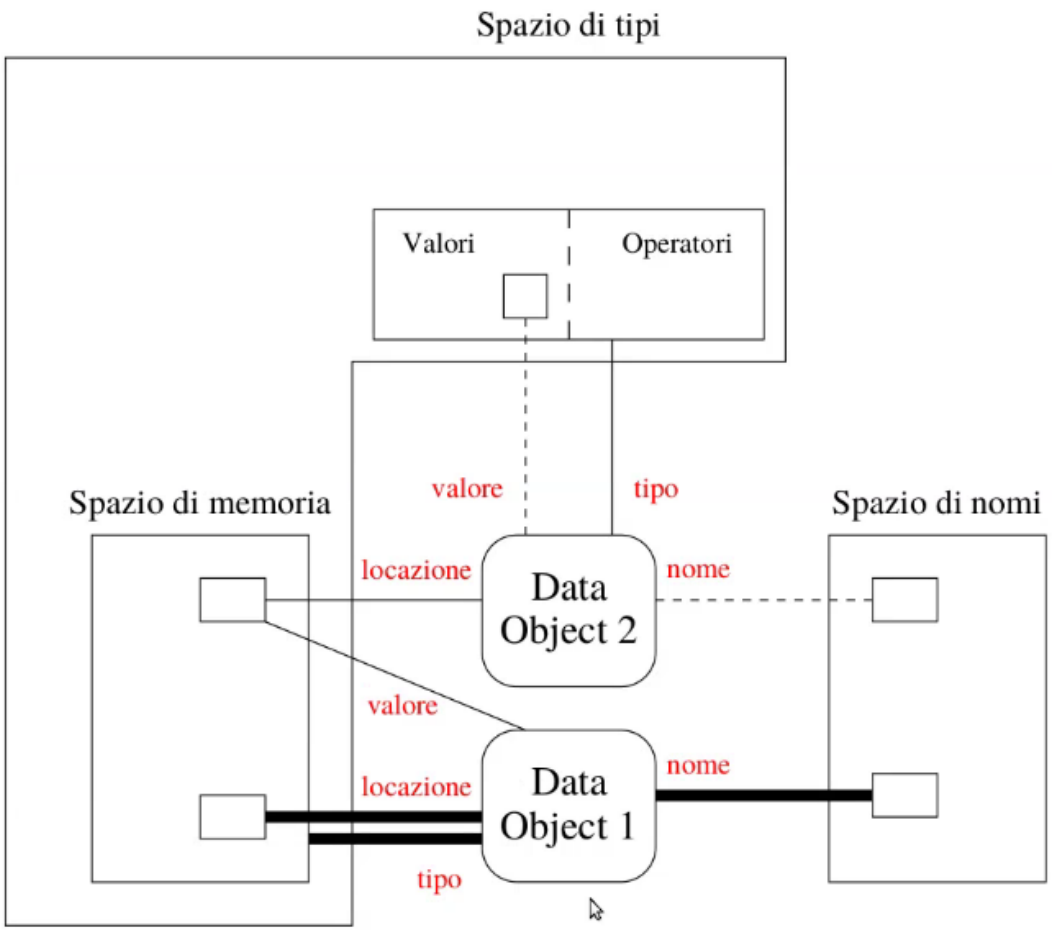
- Durante la compilazione (compile-time);
- Durante il caricamento in memoria (load-time);
- Durante L'esecuzione (run-time).

Location binding → avviene o durante il caricamento in memoria o a run-time

Name binding → avviene durante la compilazione, nell'istante in cui il compilatore incontra una dichiarazione

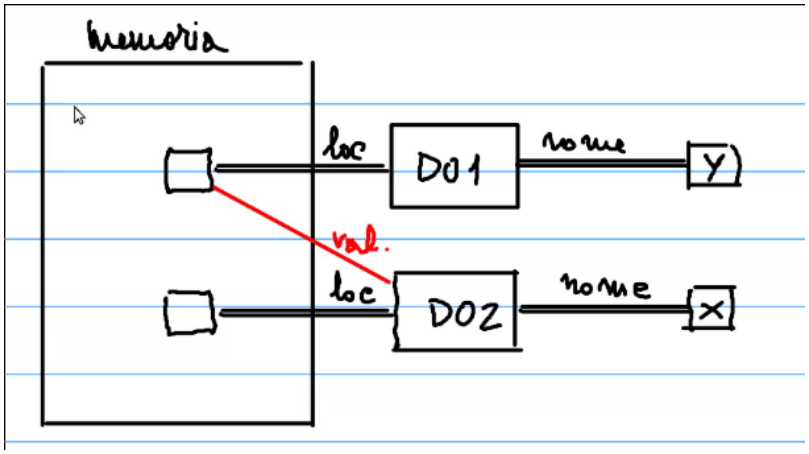
Type binding → avviene di solito durante la compilazione (nel momento in cui il compilatore incontra una dichiarazione); un tipo è definito dal sottospazio di valori che un data object puà assumere

1. I puntatori



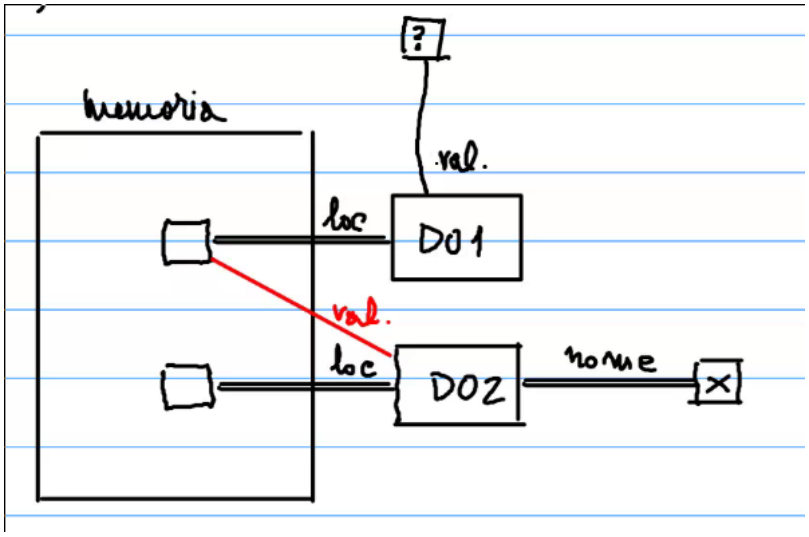
es

```
int y, *x; // CREA 2 DATA OBJECT
```




```
x = (int *) malloc(sizeof(int));
```

Il cast (int *) serve a CONTROLLA REC



2 data object coinvolti. Il secondo può non avere un legame di nome o di valore. La deallocazione è necessaria, perchè la modifica del legame di valore genera di solito dati non più accessibili per nome o riferimento

Alcuni linguaggi possiedono meccanismi di recupero automatico di memoria

5.1.2 Legame di tipo

Correlato al legame di valore. Ogni volta che il legame viene modificato occorre controllare la consistenza con il legame di tipo.

Un linguaggio è *dinamicamente tipato* se il legame (e le variazioni di legame) e di conseguenza anche il controllo di consistenza avvengono durante l'esecuzione (*es* linguaggi di scripting e Python).

Un linguaggio è *staticamente tipato* se il legame avviene durante la compilazione; in questo caso il controllo di consistenza può avvenire in entrambe le fasi.

5.1.3 Type checking

Mecanismo di controllo di consistenza della coppia valore-tipo. Avviene:

- Durante la compilazione;
- Durante l'esecuzione;

Un linguaggio è *fortemente tipato* se il controllo di consistenza avviene sempre (*es* Java).

Un linguaggio è *debolmente tipato* se il controllo di consistenza può non avvvenire affatto in numerosi casi (*es* C). Un esempio in C sono le union:

```
#include <cstdio>
union numero{
  int numero_piccolo;
  double numero_grande;
} numeri;

int main(){
  numeri.numero_grande = 3.0;
  printf("%d\n", numeri.numero_piccolo);
  printf("%f\n", numeri.numero_grande);
}
```

0
3.0

Si inserisce un double ma si legge come fosse un int → viene stampato 0 (il compilatore non segnala l'errore).

Essendo esposte a questi tipi di errori si tende a non usarle.

5.1.4 Il linguaggio perfetto

Sarebbe conveniente se il type checking avvenisse completamente durante la compilazione e in cui il compilatore non generasse più errori del necessario. Questo linguaggio sarebbe ovviamente fortemente tipato.

Questo linguaggio però non può esistere. Se esistesse il compilatore per essere capace di decidere la correttezza dell'ultima assegnazione in un caso del tipo:

```
int x;
P;
x = " pippo";
```

Dove P è un generico programma, dovrebbe essere capace di decidere la terminazione di P (l'istruzione errata eseguita se e solo se la chiamata a P termina).

Pertanto questo linguaggio non sarebbe Turing completo, contro l'ipotesi.

I compilatori nella situazione precedente ignorano se una istruzione viene eseguita o meno e segnalano comunque un errore nell'ultima linea.

```
program wrong (input, output)
var i: integer;
begin
    if false then i:=3.14;
    else i :=0;
end.
```

5.1.5 Strategie

La strategia per i linguaggi fortemente e staticamente tipati è:

- Fare più controlli possibili a tempo di esecuzione;
- Eseguire i restanti a tempo di esecuzione.

Prima si scoprono gli errori e più il testing diventa economico.

5.2 Blocchi di istruzioni

Istruzioni raggruppate in blocchi per meglio definire:

- Ambito delle strutture di controllo;
- Ambito di una procedura;
- Unità di compilazione separata;
- Ambito dei legami di nome;

I blocchi che definisco l'ambito di validità di un nome contengono due parti:

- Sezione di dichiarazione del nome;
- Sezione che comprende gli enunciati sui quali ha validità il legame.

```
...
BLOCK A;
    DECLARE I;
BEGIN A
...
END A
```

5.2.1 Ambito di validità di legami

Essenzialmente due tipi di ambito di validità (scoping):

- ambito statico o lessicale (linguaggi moderni)
  - blocchi annidati vedono e usano i legami dei blocchi più esterni (di solito aggiungon legami locali o sovrapporne di nuovi)
- ambito dinamico (linguaggi come lisp)
  - assume il proprio senso maggiore quando vi sono procedure chiamanti e chiamate (la chiamata vede e usa i legami visti e usati dalla procedura chiamante)

5.2.2 Legami di nome

```
PROGRAM P;
    DECLARE X;
BEGIN P
... {X da P}
    BLOCK A;
DECLARE Y;
    BEGIN A
    ... {X da P, Y da A}
BLOCK B;
    DECLARE Z;
BEGIN B
... {X da P, Y da A, Z da B}
END B;
```

```
... {X da P, Y da A}
    END A;
    ... {X da P}
    BLOCK C;
DECLARE Z;
    BEGIN C
    ... {X da P, Z da C}
    END C;
... {X da P}
END P;
```

5.2.3 Mascheramento

```
PROGRAM P;
    DECLARE X, Y;
BEGIN P
... {X e Y da P}
    BLOCK A;
DECLARE X, Z;
    BEGIN A
    ... {X e Z da P, Y da P}
    END A;
    ... {X e Y da P}
END P;
```

5.2.4 Legami di locazione

Si dice allocazione statica quando le variabili conservano il proprio valore ogni volta che si rientra in un blocco (il legame di locazione è fissato e costante al tempo di caricamento)

```
PROGRAM P;
    DECLARE I;
BEGIN P
    FOR I :=1 TO 10 DO
    BLOCK A;
DECLARE J;
    BEGIN A
IF I=1 THEN
    J:=1; {I da P, J da A}
ELSE
```

```
J:= J*I;
END IF
    END A;
END P;
```

Si dice allocazione dinamica di memoria quando il legame di locazione (e anche di nome) è creato all'inizio dell'esecuzione di un blocco e viene rilasciato a fine esecuzione.

È realizzata attraverso il record di attivazione di un blocco → contiene tutte le informazioni sull'esecuzione del blocco necessarie per riprendere l'esecuzione dopo che essa è stata sospesa.

Il record di attivazione puà contenere informazioni complesse, ma per realizzare un legame dinamico di locazione in blocchi annidati è sufficiente che contenga le locazioni dei dati locali più un puntatore al record di attivazione del blocco immediatamente più esterno

5.2.5 Stack di attivazione

In ogni momento dell'esecuzione lo stack di attivazione contiene i record "attivi":

- Il top dello stack contiene sempre il record del blocco correntemente in esecuzione;
- Ogni volta che si entra in un blocco, il record di attivazione del blocco viene posto sullo stack (push);
- ogni volta che si esce da un blocco, viene eliminato il record al top dello stack (pop).

es

```
PROGRAM P;
    DECLARE I, J;
    BEGIN P
BLOCK A;
    DECLARE I,K;
BEGIN A
    BLOCK B;
DECLARE I,L;
    BEGIN B
... {I e L da B, K da A, J da P}
    END B;
    ... {I e K da A, J da P}
END A;
BLOCK C
    DECLARE I, N;
```

```
BEGIN C
    ... {I e N da C, J da P}
END C;
... {I e J da P}
END P
```

6 Lezione del 19-03

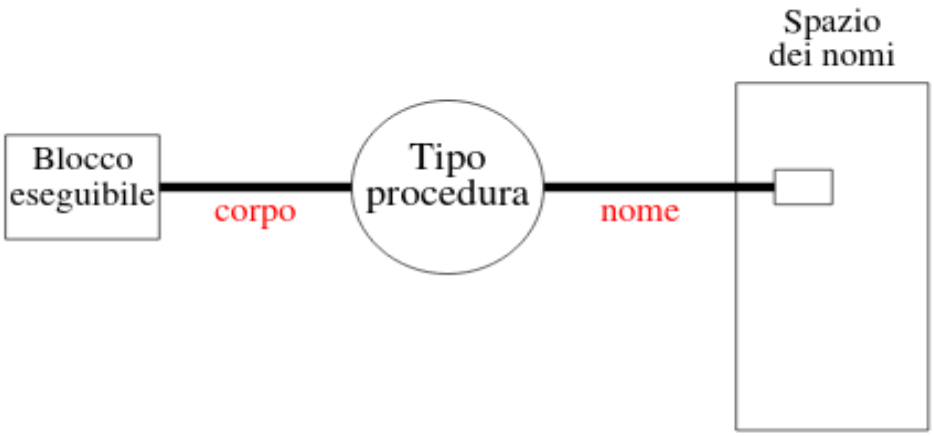
6.1 Procedure come astrazioni

Sono astrazioni di parti di programma in unità di esecuzione più piccole, come enunciati o espressioni, in modo da nascondere i dettagli ai fini del loro riuso Vantaggi:

- programmi più facili da scrivere, leggere e modificare;
- Unità di programmi indipendenti o con dipendenze benspecificate a livello più alto;
- Riusabilità del codice.

Se si distinguono come unità di esecuzione, in ordine crescente di complessità, espressioni, enunciati, blocchi, programmi, allora si definisce:

- astrazione procedurale la rappresentazione di una unità diesecuzione attraverso un'altra unità più semplice;
- Rappresentazione di un blocco attraverso un enunciato o un espressione;
- Causa la generazione di un oggetto analogo al Type Object;
- L'invocazione avviene durante l'esecuzione;
- Ogni invocazione diversa della stessa procedura causa lagenerazione di un nuovo "oggetto procedura" con lo stesso legame di tipo, ma con diverso record di attivazione.



6.2 Record di attivazione

Il record di attivazione rappresenta l'intero ambiente di esecuzione di una procedura. Esso consiste di solito in:

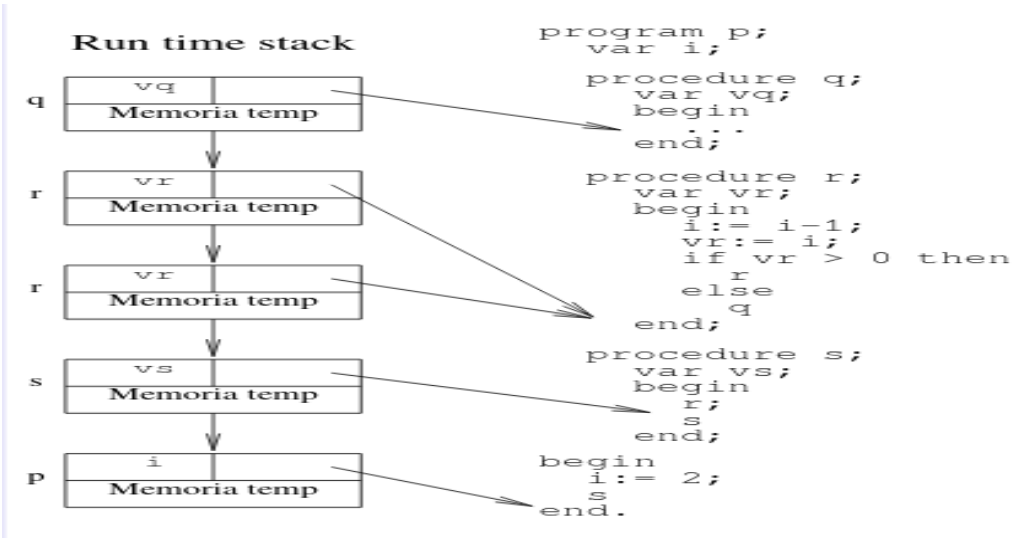
1. ambiente locale (tutti i data object che sono definiti all'internodella procedura);
2. ambiente non locale (tutti i data object la cui definizione'epropagata da altre procedure);
3. ambiente dei parametri (contiene informazioni sui dati che sonopassati [d]alla procedura);

Ogni volta che una procedura viene invocata il suo record di attivazione viene aggiunto al cosiddetto stack di esecuzione. Sul top dello stack c'è sempre il record relativo alla procedura correntemente in esecuzione. Alla terminazione della procedura, il record di attivazione viene rimosso dallo stack.

6.2.1 Ambiente locale

Include:

- Tutte le variabili dichiarate localmente;
- Il puntatore alla prossima istruzione;
- Memoria temporanea necessaria alla valutazione delle espressioni contenute nella procedura (altamente dipendente dalla realizzazione)



6.2.2 Propagazione dei Data OBJECT

Viene realizzato aggiungendo al record di attivazione un puntatore al record di attivazione della procedura da cui vengono propagate le definizioni o i dati.

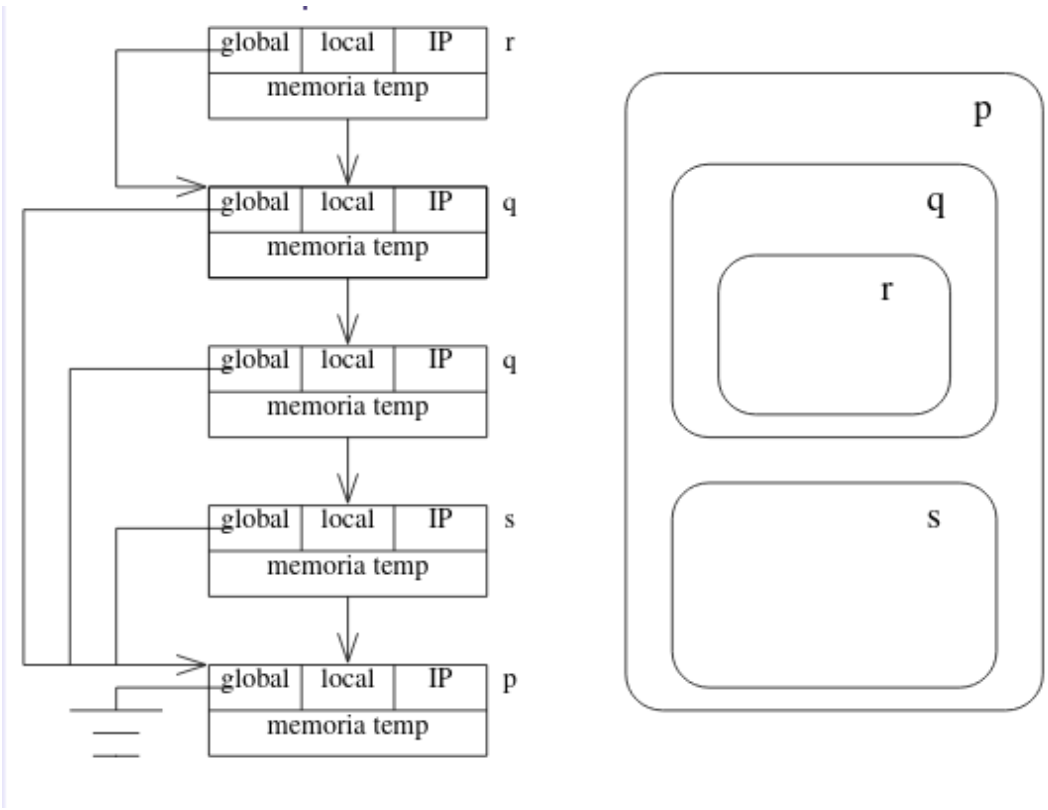
Se viene richiesto l'accesso ad un dato che non è definito localmente esso viene ricercato in modo ricorsivo nei record di attivazione precedenti. Tre tipologie di realizzazioen:

- Propagazione in ambito statico;
  - l'ambiente nonlocale di una procedura è propagato dal programma che la contiene sintatticamente → propagazione di posizione.
- Propagazione in ambito dinamico;
  - l'ambiente nonlocale di una procedura è propagato dal programma chiamante
- Nessuna Propagazione
  - l'uso di ambienti non locali è scoraggiato perchè produce effetti collaterali non facilmente prevedibili

Propagazione in ambito statico

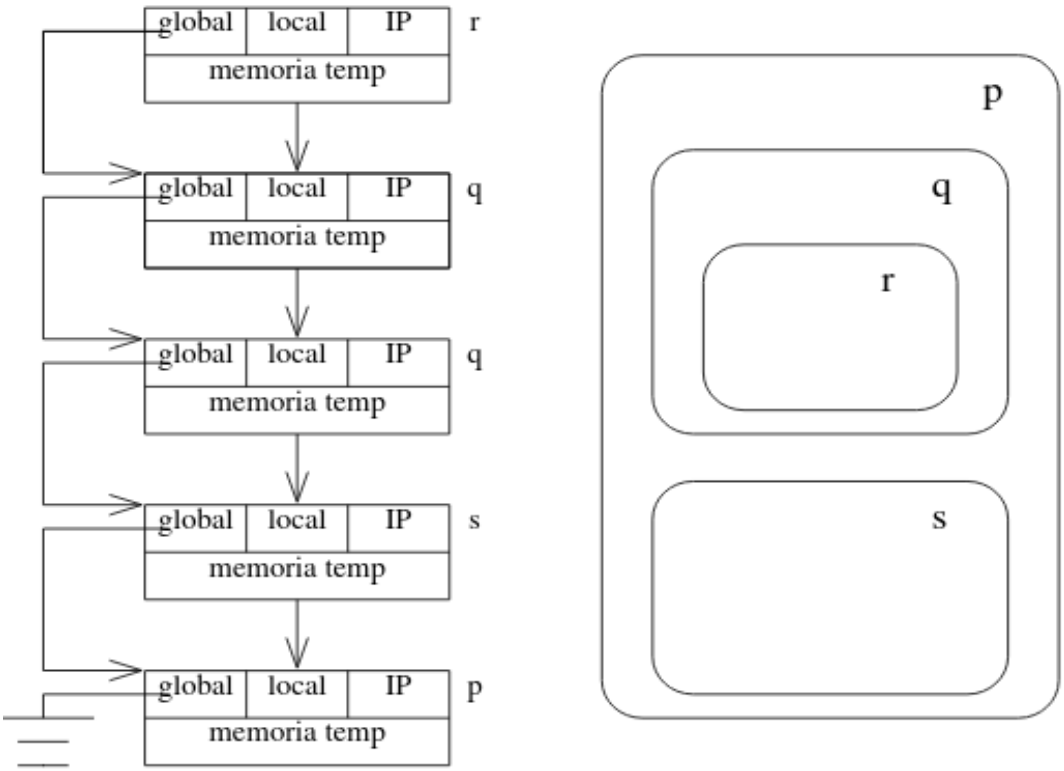
```
program p;
  var a, b, c: integer;

  procedure q;
    var a, c; integer;
    procedure r;
      var a: integer;
      begin {r}          {variabili: a da r; b da p; c da q;
        ...               procedure: q da p; r da q}
      end;  {r}
    begin {q}            {variabili: a da q; b da p; c da q;
       ...                procedure: q ed s da p; r da q}
    end;  {q}

  procedure s;
    var b: integer;
    begin {s}            {variabili: a da p; b da s; c da p;
      ...                 procedure: q ed s da p}
    end;  {s}

begin {p}                {variabili: a, b, c da p;
  ...                     procedure: q, s da p}
end.  {p}
```

Supponendo una sequenza di attivazione (p,s,q,q,r), lo stack di esecuzione ha questa forma:



Ambito dinamico

La stessa sequenza di attivazione precedente (p,s,q,q,r), generaallora lo stack di esecuzione:



Nella propagazione in ambito dinamico

- Il puntatore all'ambiente non locale non è più necessario;
- È praticamente impossibile la determinazione dell'ambiente di esecuzione di una procedura durante la scrittura del codice sorgente.

6.3 Parametrizzazione di procedure

Si distinguono in:

- parametri IN → sono passati dall'unità chiamante all'unità chiamata al momento dell'invocazione;
- parametri OUT → sono passati dall'unità chiamata alla unità chiamante al momento della terminazione della prima;
- parametri IN-OUT → transitare le informazioni in entrambe le direzioni.

Devono essere specificati in due punti:

- definizione della procedura → *parametri formali*;

- invocazione della procedura → *parametri attuali.*

Nella definizione deve essere specificato il tipo dei parametri formali. Nella invocazione è richiesta la corrispondenza di tipo tra parametri formali e attuali.

Eccezione comune:

- lasciare i parametri formali senza alcun legame di tipo → il legame si instaura durante l'esecuzione (run time) allo stesso tipo dei parametri attuali (impossibile il type checking in compilazione);
- array a dimensione variabile; il legame di tipo (e l'eventuale controllo di consistenza) verrà realizzato durante l'esecuzione.

Metodi di associazione:

- *per posizione* → a seconda della posizione relativa nella sequenza dei parametri;
- *per nome* → il nome del parametro formale è aggiunto come prefisso al parametro attuale;

es

procedure TEST (A: in Atype; b: in out Btype; C: out Ctype) – invocazione che usi associazione per posizione è: TEST(X, Y, Z); – mentre una che usi associazione per nome può essere TEST(A=>X, C=>Z, b=>Y);

Una tecnica è la cosidetta *associazione di default.* Essa permette di specificare valori di default ai parametri formali che non sono stati legati a valori da parametri attuali.

6.3.1 Parametri IN

Realizzati in 2 modi:

- Con un riferimento → la locazione del parametro attuale diventa la locazione del parametro formale
  - poichè il parametro formale è di tipo IN, allora si deve impedire la modifica all'interno della procedura;
- Con una copia → si alloca il parametro formale all'interno dello stack di attivazione
  - parametro formale e attuale presentano sono allocate in due posizioni diverse;
  - modifica permessa, perchè valida solo nell'ambiente diesecuzione della procedura;

Il secondo modo è meno efficiente del primo, sia rispetto allo spazio sia al tempo, ma è più flessibile e richiede meno variabili locali.

6.3.2 Parametri OUT

Realizzati in 2 modi:

1. Con un riferimento;
2. Con una copia.

Rappresentano risultati → alcuni linguaggi assumono che i parametri OUT non siano inizializzati e ne proibiscono la "lettura", ad es:

- Uso a destra di un assegnamento;
- Passaggio a un parametro IN o IN-OUT di un'altra procedura.

6.3.3 Parametri IN-OUT

Combinazione dei due precedenti. Realizzati in 2 modi:

- Con un riferimento;
- Con una copia.

6.4 Aliasing

È la possibilità di riferirsi alla stessa locazione con nomi diversi. Nel passaggio dei parametri può causare notevoli problemi di interpretazione.

6.5 Procedure come parametri di procedure

Alcuni linguaggi permettono l'uso di procedure come argomento di altre procedure.

7 Lezione del 23-03

7.1 Memo

In ADA sono presenti le keyword *in, out, in out.*

Pascal o non mette nulla (passaggio per copia) o utilizza la keyword *var* (*in out* per riferimento)

C, C++, Java presentano solo il passaggio *in* per copia

In C e C++ si può simulare *in-out* per riferimento (sfruttando gli opportuni operatori)

In Java il passaggio di oggetti somiglia a un passaggio per riferimento (viene passato il puntatore all'oggetto).

7.2 Funzioni

Sono procedure che restituiscono un valore alla procedura chiamante. Sono realizzate:

- O creando una pseudovariabile nell'ambiente locale della procedura chiamata
  - questa variabile può essere solo modificata (nessun accesso in lettura);
- O utilizzando una istruzione di return per restituire esplicitamente il controllo alla procedura chiamante inviandole allo stesso tempo il valore di una espressione

8 Lezione del 26-03

8.1 Esempi sul record di attivazione nei linguaggi

8.1.1 Esempio interferenza

```
#include <stdio.h>
void interferenza(int x){
    int var;
    printf("var = %d\n", var);
    var = x;
    }

void interferenza_bis(int x){
    int var;
    printf("var = %d\n", var);
    var = x;
}
```

```
int main (void){
    interferenza(1);
    interferenza_bis(2);
    interferenza(3);
}
```

$$\begin{aligned} \text{var} &= 0 \\ \text{var} &= 1 \\ \text{var} &= 2 \end{aligned}$$

8.1.2 Esempio copia

```
#include <stdio.h>
void copy(char x){
  char buf[5];
  int i=0;
  do{
  buf[i] = x[i];
  }
  while(x[i++] != '\0');
}
int main(void){
    char v[] = "abcdef";
    copy(v);
    return 0;
}
```

Si ottiene uno `stack mashing detected` (si cerca di inserire dati in quantità maggiore di quanti ne possa contenere il `buf[5]`)

8.2 Macro

Generazione di un nuovo brano di codice sorgente (espansione della macro) in cui i nomi dei parametri attuali sostituiscono i nomi dei parametri formali. *es.* Data la procedura

```
procedure swap (a, b: integer);
var temp: integer;
begin
    temp := a;
    a := b;
```

```
    b := a;
end;
```

La chiamata *swap(x,y)* esegue il seguente brano di codice:

```
temp := x;
x := y;
y := temp;
```

8.3 Implemantazione efficiente dello scoping statico

L'accesso ad una variabile non è istantanea → bisogna scandire una lista che potrebbe essere lunga.

8.4 Annidamento

```
program p;
var a,b,c int;
    procedure q;
    var a,c int;
procedure r;
var a int;
    procedure s;
    var b int;
```

Una procedura q è annidata in un blocco b se è definita dentro b, ad *es*:

- q e s sono annidate in p;
- r è annidata in q.

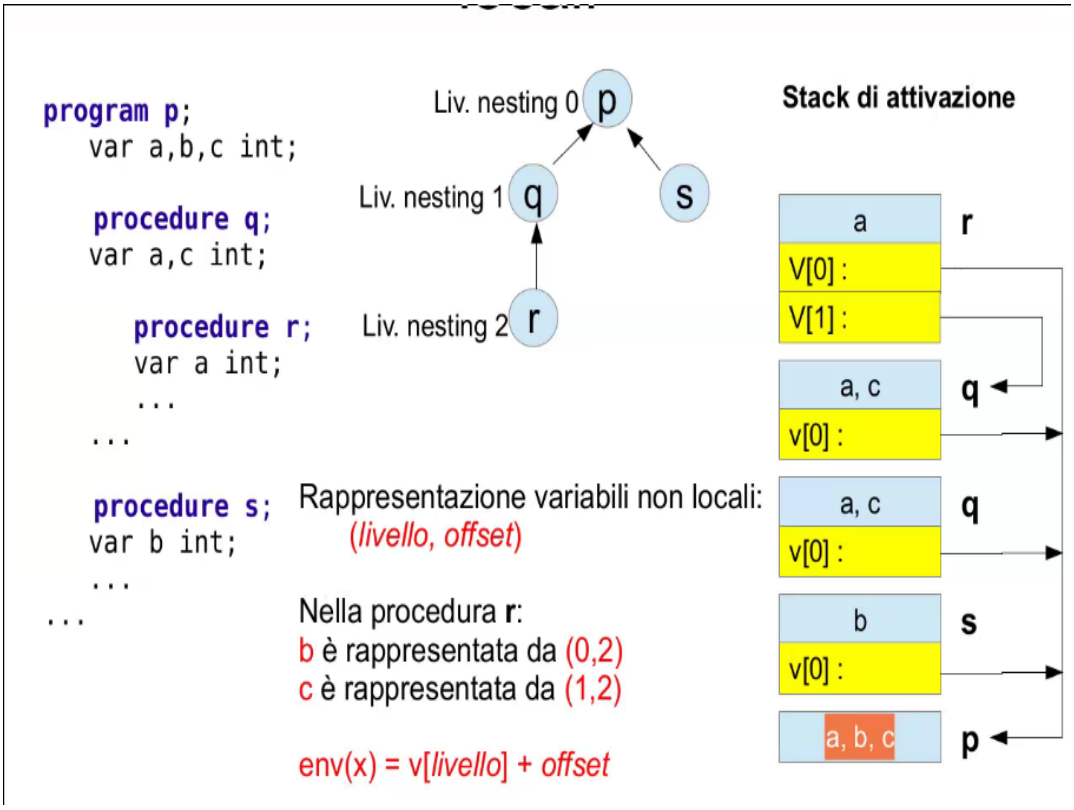
Il livello di nesting di una procedura è il numero di blocchi che a contengono:

- il livello di nesting di q ed s è 1;
- Il livello di nesting di r è 2.

L'ambiente statico non locale di una procedura è dato dall'ambiente delle procedure in cui è innestata

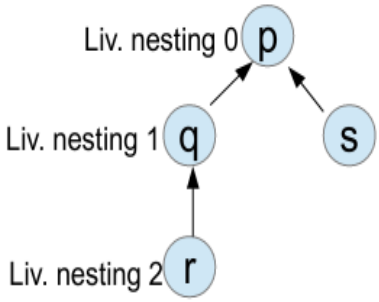
L'annidamento si può rappresentare come un albero. I livelli dell'albero di nesting corrispondono ai livelli di nesting

8.5 Rappresentazione variabili non locali



8.6 Mantenimento del vettore degli ambienti non locali

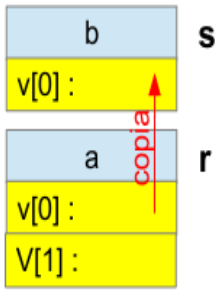
```
program p;
   var a,b,c int;

   procedure q;
   var a,c int;

      procedure r;
      var a int;
      ...
   ...

   procedure s;
   var b int;
   ...
...
```



Chiamata a procedure definite esternamente:

Esempio 2: r chiama s:

- il liv. di nesting **decresce**
- Copiare solo gli elementi di livello *minore o uguale* a quello di s
- Struttura ad albero + scoping statico garantiscono sempre di trovare l'ambiente non locale della procedura chiamata nel record del chiamante



8.7 Proprietà di questa implementazione

- Accesso alle variabili in tempo costante
  - Accesso a vettore + somma del puntatore ivi contenuto e dell'offset;
  - Indipendente dal livello di nesting;
  - Calcolo supportato direttamente dalle istruzioni macchina.
- Creazione record di attivazione lineare nel livello di nesting (copia degli elementi di v[])
  - Indipendente dall'esecuzione;
  - Dipende solo dal testo del sorgente;
- Tempo costante;
- L'implementazione naive richiederebbe a run time di percorrere le liste di puntatori all'ambiente non locale
  - Accesso alle variabili in tempo **lineare** nel livello di nesting;
  - Creazione dei record di attivazione in tempo **lineare** nel livello di nesting.

8.8 Paradigma OO

Rappresentazione di ogni cosa in un programma OO.

In essenza → progettare un sistema OO consiste nell'identificare quali oggetti il sistema debba contenere, il loro comportamento e le responsabilità di ognuno di essi e come essi interagiscono tra loro

8.9 Encapsulation

Le strutture dati che implementano i valori sono accessibili direttamente solo dalle operazioni fornite dal tipo → è una regola di **scoping**

8.10 C++ vs Java

C++

Acquisisce eredità di C ed è compatibile con programma C esistenti.

Cresciuto in modo abnorme ed è veramente difficile raggiungere piena competenza in tutti gli aspetti del linguaggio.

Java

WWW e abilità di eseguire *web applet* su qualunque elaboratore e SO.

Linguaggio ben progettato con pochi costrutti. Risolti i prooblemi della velocità di esecuzione.

Diffusamente utilizzato nell'ambito educativo

8.11 UML

Linguaggio i cui simboli sono diagrammi

8.12 Essenza di un oggetto

- Insieme di **attributi** (valore, stato interno, o qualunque altra cosa sia necessaria per il modello dell'oggetto);
- **Abilità** di modificare lo stato degli attributi;
- **Responsabilità** sotto forma di servizi offerti ad altri oggetti.

  Gli oggetti esterni non hanno conoscenza di come un oggetto realizza gli attributi internamente, ma piuttosto devono conoscere quali servizi un oggetto offre.

8.13 Classe

Descrizione di un insieme di oggetti che condivdono attributi e comportamento comuni. Simile a quello del tipo di dato astratto nei linguaggi non OO

La definizione di una classe descrive tutti gli attributi comuni agli oggetti della classe, così come tutti i metodi che realizzano il comportamento comune degli oggetti della classe.

Gli oggetti membri di una classe sono chiamati **istanze** di quella classe → I membri di una classe variano durante l'esecuzione di un programma

8.14 Polimorfismo

Permette al mittente di ignorare l'identità (il tipo) del ricevente.

Realizzato con un mix di overriding e upcasting.

9 Lezione del 30-03

9.1 Classi astratte

Definiscono una interfsaccia comune a tutte le sottoclassi, specificando solo i nomi (la firma) dei metodi che le sottoclassi "concrete" dovranno definire.

Poichè sono incomplete della realizzazione dei metodi, le classi astratte non possono avere istanze.

Una classe che estende una classe astratta eredita tutti i suoi metodi (inclusi quelli astratti). Ogni classe che eredita metodi astratti è considerata astratta essa stessa, a meno che essa nonrealizzi tutti i metodi astratti ereditati

9.2 Visibilità delle componenti di una classe

È la possibilità di una classe di veere ed usare le risorse di un'altra classe. Livello di visibilità:

- private (`-`) → attributi e metodi sono visibili solo a oggetti istanze della stessa classe;
- Package (`#`) → attributi e metodi sono visibili ad un insieme specifico di classi;
- Public (`+`) → attributi e metodi sono visibili a tutti gli altri oggetti.

Normalmente si cerca di evitare attributi publici. Se si deve reperire o modificare le informazioni sullo stato interno di uno oggetto, lo si fa utilizzando i metodi specifici (e visibili) dell'oggetto

9.3 Ereditarietà

Il concetto di ereditarietà descrive come si possano condividere attributi e funzionalità tra classi di natura o scopo simile.

Vi sono due modi con cui si può aggiungere il concetto di ereditarietà al modello del sistema:

- Generalizzazione;
- Specializzazione.

9.3.1 Generalizzazione

Avviene quando si riconosce che più classi dello stesso diagramma Class esibiscono funzionalità,struttura, scopo comuni.L'analista decide allora di creare una nuova classe contenente gli attributi e le funzionalità comuni esemplifica le classi precedenti come estensioni della nuova(generalizzata).

9.3.2 Specializzazione

Avviene quando si riconosce che la nuova classe che si sta aggiungendo ha le stesse funzionalità,struttura e scopo di una classe esistente, ma ha bisogno di nuovo codice o attributi.La nuova classe è una forma *specializzata* di quella già esistente.

9.4 Associazioni e molteplicità

9.4.1 Associazioni complesse

Sono quelle associazioni in cui vi sono molteplicità maggiori di 1 in entrambe i ruoli. Difficile da implementare. 2 possibili soluzioni:

- Classe di associazione
  - Il nome della classe deve essere lo stesso della associazione;
  - Una classe di Associazione è usata per documentare la risoluzione di una associazione complessa, non fa parte delle astrazioni chiave;
  - Restano nei diagrammi Class finchè, nella fase di progettazione, non si decide di realizzarle;
- Associazione qualificata
  - Usate quando si vuole evidenziare che la realizzazione futura sarà mediante un array, una hash table, un dizionario;

9.4.2 Ruoli nelle associazioni

- Riferiscono ad una associazione tra classi;

- Etichette poste alle estremità di una associazione;
- Rappresentano un attributo all'interno della classe che si trova all'estremità opposta del nome.

9.4.3 Associazioni riflessive

In un diagramma Object possono esistere link tra istanze della stessa classe, ma c'è una sola classe nel diagramma Class per rappresentare entrambi gli oggetti. In tal caso è usata una associazione riflessiva.

9.5 Introduzione a Java

Un linguaggio di programmazione, con sintassi simile a C++ e semantica simile a SmallTalk.

Le applicazioni Java sono programmi autonomi che richiedono un elaboratore su cui sia installato Java Runtime Environment(JRE).

Un ambiente di sviluppo, cioè un insieme di numerosi strumenti per la realizzazione di programmi: un compilatore, un interprete, un generatore di documentazione, uno strumento di aggregazione di file, etc...

9.5.1 Errori comuni

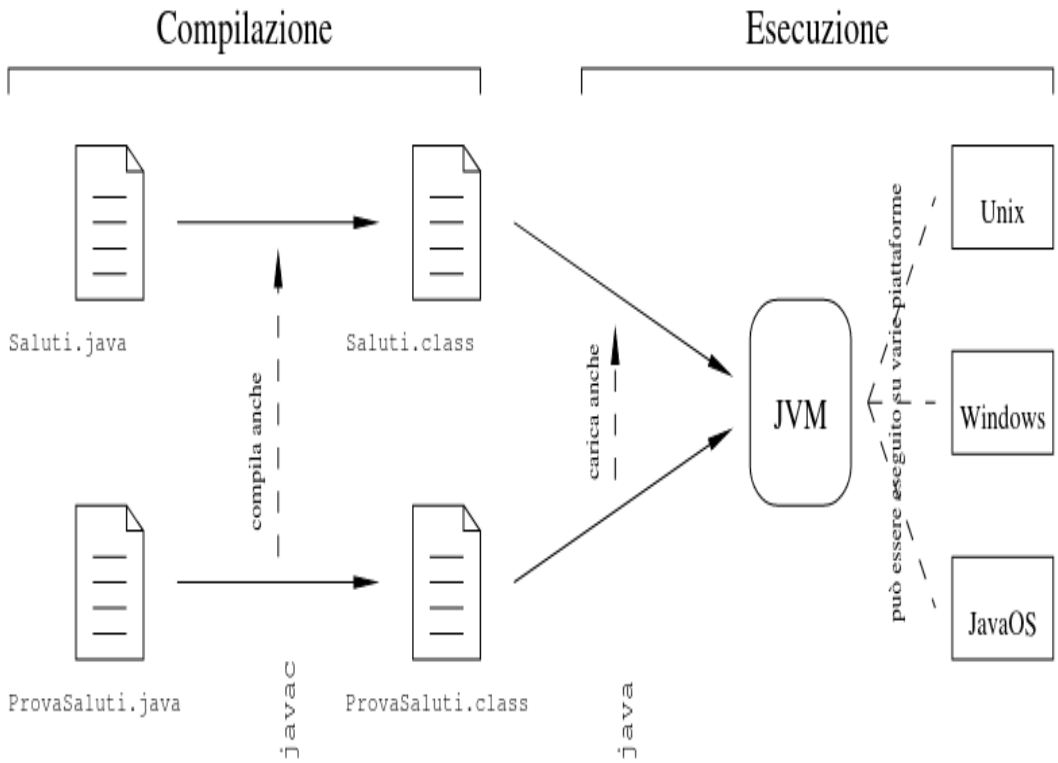
Errori in compilazione:

- Installazione JavaSDK non corretta;
- Nome di metodi di sistema non corretti;
- Errata corrispondenza tra nome di classe e nome di file;
- Numero di classi pubbliche in un file;

Errori in esecuzione:

- Errata scrittura della classe da caricare inizialmente in memoria;
- Errata scrittura della firma del metodo main;

9.5.2 Compilazione ed Esecuzione



9.5.3 Sicurezza

Miglioramento della sicurezza del codice che proviene da fonti esterne.

Il Class Loader:

- Carica solo le classi necessarie per l'esecuzione del programma;
- Mantiene le classi del file system locale in uno spazio di nomi separato
  - limita applicazioni "cavallo di Troia", poichè le classi locali sono sempre caricate per prime;
- previene il così detto *spoofing*.

Il Bytecode Verifier controlla che:

- Il codice rispetti le specifiche della JVM;
- Il codice non violi l'integrità del sistema;
- Il codice non generi stack overflow o underflow;
- Non vi siano conversioni di tipo illegali (*es* la conversione di interi in puntatori).

9.5.4 Costruttori

NB:

- I costruttori NON sono metodi;
- I costruttoni NON sono ereditati;
- Un costruttore NON ha valore di ritorno;
- Gli unici modificatori validi per il costruttore sono public,protected e private.

Costruttore di default

C'è sempre almeno un costruttore in ogni classe. Se il programmatore non scrive nessun costruttore, allora verrà usato un costruttore di default.

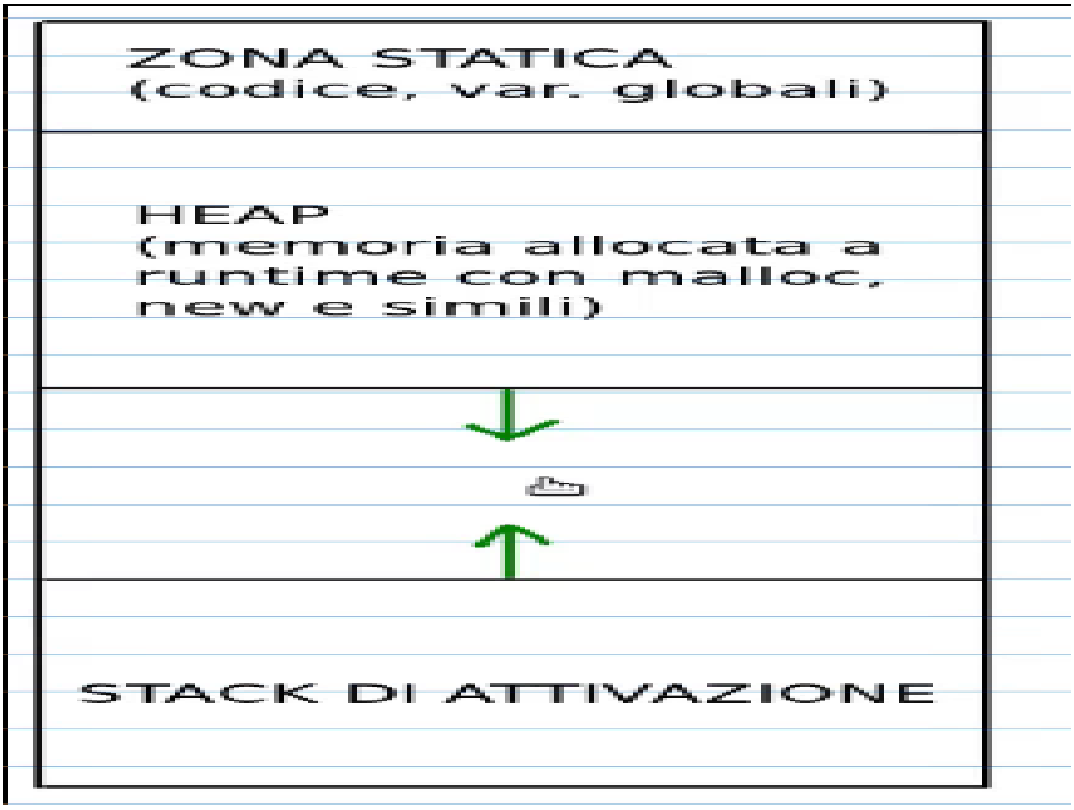
Il costruttore di default non ha argomenti, non ha corpo.

Permette di creare istanze di classi (con l'espressionenew Xxx()) senza dover scrivere un costruttore.

NB → Se si aggiunge una dichiarazione di costruttore con argomenti ad una classe che in precedenza non aveva costruttori espliciti, allora si perde il costruttore di default. Da quel punto in poi, una chiamata a *new Xxx()* genererà un errore di compilazione.

10 Lezione del 09-04

10.1 Organizzazione della memoria nei linguaggi di programmazione



10.2 Ambiente non locale nel paradigma a oggetti

- Consiste negli attributi dell'oggetto che esegue il metodo;
- È ancora una forma di scoping statico
  - Attributi dichiarati nel blocco di classe che contiene il metodo;
- Allocato nello heap;
- Viene indicato esplicitamente
  - con il destinatario del messaggio;
  - *this* se il destinatario non viene specificato.

10.3 Java

10.3.1 Commenti

Stessa logica del `C/C++`.

10.3.2 Blocchi

Suddivisi o su più righe o racchiuse tra parentesi

10.3.3 Identificatori

- Sono nomi assegnati a variabili, classi e metodi;
- Possono cominciare con un carattere Unicode, underscore, o il dollaro;
- Sono case sensitive e non hanno lunghezza massima;
- Il nome di una classe è costituito solo da caratteri ASCII per mantenere la massima portabilità.

10.3.4 Tipi primitivi

- Logici → boolean (true e false);
- Testuali → char;
- Interi → byte, short, int, long;
- Floating point → float, double.

Non c'è cast tra tipi interi e boolean. Interpretare valori numerici come valori logici non è consentito in java.

10.3.5 Tipo String

Il tipo String non è un tipo primitivo, ma una classe (è scritto infatti con la S maiuscola). Ha i suoi litterali compresi tra apici doppi

10.3.6 Tipi Reference

Tutti i tipi primitivi sono tipi *reference*. Una variabile reference contiene la "maniglia" di un oggetto

10.3.7 Costruzione degli oggetti

`new Xxx()` serve ad allocare spazio per il nuovo oggetto. Scatena le seguenti operazioni:

- Viene allocato lo spazio per il nuovo oggetto e le variabili dell'istanza sono inizializzati al loro valore di default;
- Viene eseguita ogni inizializzazione esplicita degli attributi;
- Viene eseguito un costruttore;
- Viene assegnato il riferimento finale dell'oggetto.

10.3.8 Parametri

- Java permette il passaggio dei parametri per valori (parametri IN realizzati per copia);
- Il passaggio per riferimento (che permette la modifica del valore del parametro nel contesto della procedura chiamante) è proibito in Java;
- Quando si passa una istanza di oggetto come argomento di un metodo, quello che si sta passando non è l'oggetto, ma solo un riferimento a quell'oggetto
  - Questo riferimento è copiato nel parametro formale;
- Sarà possibile la modifica nel consteso del chiamante (mai del valore della variabile) dell'oggetto a cui fa riferimento quello variabile.

10.3.9 Il riferimento this

La parola chiave può essere usata:

- Per fare riferimento, all'interno di un metodo o di un costruttore locale, ad attributi o metodi locali
  - Questa tecnica è usata per risolvere ambiguità in alcuni casi in cui una variabile locale di un metodo maschera un attributo locale dell'ggetto;
- Per permettere ad un oggetto di passare il riferimento a se stesso come parametro ad una altro metodo o costruttore.

10.3.10 Convenzioni sul codice

- Package → package oggetti.*;
- Classi → class NomeClasse;
- Interfaccia → interface NomeInterfaccia;
- Metodi → getNomeMetodo();
- Variabli → dimensioniX;
- Costanti → `PI_GRECO`;
- Spaziatura →
  - un solo enunciato per riga;
  - due spazi di indenting per blocchi annidati.

10.4 Conversioni

All'inizio esistevano solo i tipi elementari e nessuna gerarchia di classi. Vengono in seguito introdotti i tipi user-defined.

10.4.1 Type equivalence

- Name equivalence → i tipi di x e y devono avere lo stesso nome (stesso tipo);
- Structural equivalence → i tipi di x e y devono avere la stessa rappresentazione interna
  - Più snello e flessibile ma più soggetto ad errori.

In C si punta sempre al secondo caso, tranne per le struct dove è richiesto il Name equivalence. Questo scelta deriva dalla richiesta di verifica di una proprietà detta *bisimulazione*.

`Struct S1{int a; struct S1* next;}; Struct S2{int a; struct S3* next;}; Struct S3{int a; struct S2* next;};`

- Caso semplice → sia S1 che S2 generano tutte le catene come quelle a destra;
  - Complicazioni in presenza di record varianti

10.4.2 Compatibilità di tipi

Con l'avvento dei linguaggi OO l'equivalenza viene rimpiazzata da *compatibilità*. Qualunque sottotipo di T è compatibile con T.

Nei linguaggi OO pi comuni la compatibilità è basata su *nome* piuttosto che *struttura*:

- Due classi con nomi diversi sono diverse;
- Anche se hanno gli stessi attributi e gli stessi metodi;
- Inoltre una classe per essere sottotipi di un'altra deve essere esplicitamente dichiarata tale (*extends*)
  - La struttura non conta neanche in questo caso

10.5 Variabili

Variabili definite all'interno di un metodo sono dette **locali** → create quando il metodo è invocato e distrutte alla sua terminazione.

Esse devono essere inizializzate esplicitamente prima di essere usate.

Variabili usate come parametro di metodi, visto il meccanismo di passaggio di parametri (IN per copia) sono variabili locali.

Variabili definite all'esterno di un metodo sono creat quand viene costruito un oggetto. Di 2 tipi:

- **Variabili di classe** → dichiarate usando il modificatore *static*

  - Create nel momento in cui la classe è caricata in memoria;
  - Continuano ad esistere finchè la classe resta in memoria;
  - Una variabile valida per tutte le istanze;

- **Variabili di istanza** → dichiarate senza usare il modificatore *static*
  - Create durante la costruzione dell'istanza;
  - Esistono finchè l'oggetto esiste

10.6 Inizializzazione

Nessuna variabile può essere usata prima di essere inizializzata. Le variabili non locali sono inizializzate dalla JVM, nel momento in cuo la classe è caricata in memoria o in cui è allocato spazio per il nuovo oggetto ai seguenti valori:

Variabile	Valore
byte	0
short	0
int	0
long	0L
float	0.0F
double	0.0D
char	'0̆00'
boolean	false
riferimenti	null

Le variabili locali (quelle dei metodi) devono essere inizializzate manualmente prima dell'uso.

11 Lezione del 13-04

11.1 Operatori in Java

Simili al `C/C++`.

11.1.1 Booleani

Operatori booleani → `!` `(NOT)`, `&` `(AND)`, `|` `(OR)`, `^(XOR)`; Operatori booleani con corto circuito → `||` `(OR)`, `&&` `(AND)`

11.1.2 Di bit

Operatori di manipolazione di bit su interi `~` `(Complemento a 1)`, `&` `(AND)`, `|` `(OR)`, `^(XOR)`

11.1.3 Right e Left Shift

Nello shift a destra aritmetico o con segno (`»`) il bit di segno viene conservato. Nello shift a destra logico o senza segno (`»>`) il bit di segno è azzerato nello spostamento. Se l'operando di sinistra è un `int`, prima di applicare lo shift viene ridotto l'operando di destra modulo 32; se l'operando è un `long` prima di applicare lo shift viene ridotto l'operando di destra modulo 64

x `»>` 64 non modifica il valore di x

L'operatore `»>` è ammesso solo per i tipi interi e long. Se viene usato per `short` o `byte` questi valori vengono promissi con estensione di segno, ad `int` prima di aplicare lo shift.

Per lo shift a sinistra non c'è il problema del bit di segno

11.1.4 Concatenazione di stringhe

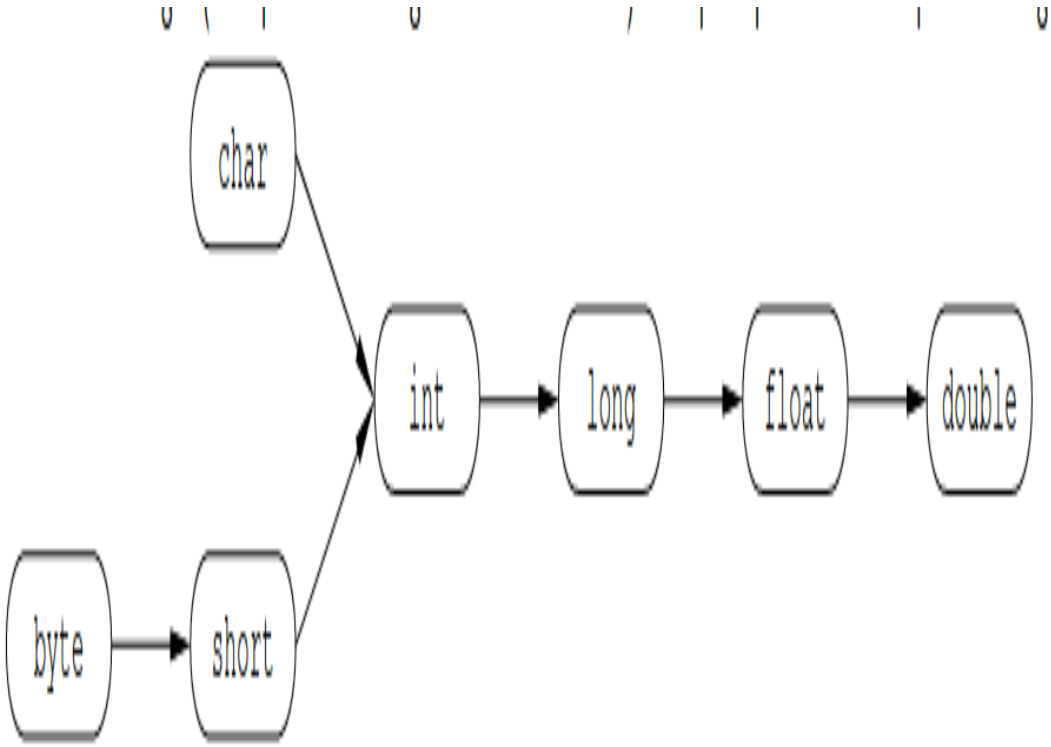
L'operatore `+` esegue la concatenazione di 2 oggetti di tipo String, producendo un nuovo oggetto di tipo String. Un argomento deve essere un oggetto String. Gli altri argomenti sono convertiti a String automaticamente, attraverso il metodo `toString`.

11.2 Casting di primitivi

Una conversione di tipo per assegnazione avviene quando si assegna un valore ad una variabile di tipo differente. Ciò può avvenire o con un operatore di assegnazione, o durante il passaggio di parametri ad un metodo.

Il tipo booleano non può mai essere convertito

11.2.1 Conversioni legali



11.2.2 Conversioni esplicite

Se è possibile la perdita di informazioni in una assegnazione o in un passaggio di parametri, il programmatore DEVE confermare l'assegnazione con un *cast* esplicito. In un cast esplicito, il valore in eccesso viene troncato.

Ad esempio l'assegnazione da `long` a `int` richiede un cast esplicito

Il casting esplicito non è necessario per i litterali interi (non i floating point) che ricadono nel range legale del tipo di destinazione.

11.2.3 Promozione aritmetica

In una espressione aritmetica, le variabili sono automaticamente promosse ad una forma più estesa per non causare perdita di informazioni.

Le regole per operatori unari:

- Se l'operando è un `byte`, uno `short`, un `char` esso è convertito ad `int`
  - (a meno che l'operatore non sia `++` o `--`, nel qual caso non avviene nessuna conversione);

- Altrimenti nessuna conversione;

Le regole per operatori binari:

- Se uno degli operandi è un `double`, allora l'altro operando è convertito ad un `double`;
- Se uno degli operandi è un `float` allora l'altro operando è convertito ad un `float`;
- Se uno degli operandi è un `~long` allora l'altro operando è convertito ad un `long`;
- Altrimenti, entrambi gli operandi sono convertiti ad `int`.

11.3 Enunciati Branch

11.3.1 If & else

Richiede una espressione booleana, non un intero come in `C/C++`

11.3.2 Switch

```
switch  (<expr >){
   case <constant1 >:
     <statement >*
       break;
   case <constant2 >:
     <statement >*
       break;
   default :<statement >*
      break;
```

Dove:

- `<expr>` → deve essere compatibile per assegnazione con `int` (char, byte, short sono promossi automaticamente);
- l'etichetta opzionale `<default>` è usata per specificare il segmento di codice da eseguire quando il valore `<expr>` non corrisponde a nessuno dei valori delle stanze case;
- Se non è presente l'enunciato opzionale `break`, l'esecuzione continua nella stanza case successiva.

11.4 Enunciati loop

11.4.1 For

```
for (<init_expr >; <bool_expr >; <alt_expr >)
 {<statement >*}
```

11.4.2 While

```
while (<bool_expr >) {
<statement >*}
```

11.4.3 `do/while`

```
do {<statement >*
} while (<bool_expr >);
```

11.5 Controlli speciali

- `break [label]` → usata per uscire in modo prematuro da un enunciato `switch`, da enunciati di loop o da blocchi etichettati;
- `continue [label]` → usata per saltare direttamente alla fine del corpo di un loop;
- `label: <statement>` → usata per identificare un qualinque enunciato verso il quale può essere trasferito il controllo.

12 Lezione del 16-04

12.1 Array

Si possono dichiarare array di tipi primitivi o di riferimenti ad oggetti.

Un array è un oggetto → le dichiarazioni precedenti creano solo il riferimento al rispettivo oggetto.

Quindi:

- Non viene qui generato l'oggetto → la generazione richiede sempre l'uso della new;
- Nella dichiarazione non si deve specificare la dimensione dell'array;
- Se si dichiarano più array di uno stesso enunciato usando le parentesi quadre a sinistra, le parentesi sono applicate a tutte le variabili alla loro destra

`int a[], b` → a è riferimento ad array, b è int;

`int [] a, b` → a e b entrambi riferimenti ad array

12.1.1 Creazione

Per la creazione dell'array si usa sempre la `new`.

12.1.2 Inizializzazione

Poichè l'inizializzazione delle variabili è cruciale, Java offre 2 metodi abbreviati per gli array.

Il primo consiste nella dichiarazione, costruzione ed inizializzazione in una riga

```
String [] nomi = {
    "Antonio",
    "Marcello",
    "Anna"
}
// equivalente a
String [] nomi;
nomi = new String[3];
nomi[0] = "Antonio";
nomi[1] = "Marcello";
nomi[2] = "Anna";
```

Il secondo metodo è la costruzione ed inizializzazione di un array anonimo:

```
int [] esempio1;
esempio1 = new int[] {4,7,3};

// O ancora meglio
  public class A{

      void prendiArray(int [] unArray){
// usa unArray
      }
      public static void main(String[] args){
A a = new A();
a.prendiArray(new int[] {3,4,5,6,7});
      }
  }
```

In java tutti i parametri dei metodi sono di tipo IN (realizzati) per copia. Questo significa che il valore del parametro nel metodo chiamante non può essere modificato. Tuttavia, è possibile creare un riferimento a tale parametro e realizzare ugualmente la modifica nel metodo chiamante.

12.1.3 Multidimensioni

- Array di array;
- Array non rettangolare di array;
- Array rettangolare di array.

Costruzione e inizializzazione → `int [][] multiD = {{5,4,3,2}, {9,8}, {7,6,5}};`

12.1.4 Estremi

- Indice iniziale 0;
- Numero di elementi è parte dell'array, nell'attributo lenght;
- Accesso oltre i limiti causa il lancio di una eccezione a runtime.

12.1.5 Assegnazione

In una assegnazione l'array deve avere lo stesso numero di dimensioni della variabile di riferimento.

12.1.6 Ridimensionamento

Gli array non sono ridimensionabili. Si può usare la stessa variabile per riferirsi ad un nuovo array.

12.1.7 Copia

Si utilizza il metodo `System.arraycopy()`.

Questo metodo copia i valori contenuti negli elementi dell'array → nel caso di array di oggetti (o di array multidimensionali), ciò significa che vengono copiati i riferimenti agli oggetti, non vengono cioè create nuove copie di oggetti.

12.1.8 Aiutare il GC

Il garbage collector non è onnipotente → necessita a volte di un piccolo aiuto. Infati se l'utilizzatore del metodo abbandona il valore di ritorno ricevuto, esso non sarà eleggibile per GC, finchè il riferimento ad esso non sarà sovrascritto. Questo potrebbe richiedere tempi lunghissimi.

12.2 Ereditarietà

12.2.1 Specializzazione

- Una sottoclasse eredita tutti i metodi e le variabili della superclasse;
- Una sottoclasse non eredita i costruttori della superclasse;

- I 2 modi (esclusivi uno con l'altro) per includere i costruttori in una classe sono
  - Usare il costruttore di default (senza parametri);
  - Scrivere 1 o più costruttori espliciti.

12.3 Polimorfismo

- Un oggetto ha una sola forma;
- È l'abilità di riferirsi con la medesima variabile ad oggetti con forme diverse (credendo, però di riferirsi ad un oggetto di una particolare forma ed ignorando, cioè la reale forma dell'oggetto);

12.4 Collezioni eterogenee

12.4.1 Collezioni omogenee di oggetti dela stessa classe

```
MiaData[] date = new MiaData[2];
date[0] = new MiaData(5,5,2006);
date[1] = new MiaData(25,12,2006);
```

12.4.2 Collezioni eterogenee di oggetti di classi diverse:

```
Impiegato[] staff = new Impiegato[100];
staff[0]  = new Impiegato();
staff[1]  = new Impiegato();
staff[2]  = new Quadro();
```

12.5 Argomenti polimorfici

- Si possonno costruire metodi che accettino come parametri un riferimento "generico" ed operare in modo automatico su un più vasto insieme di oggetti;
- In questo caso è il metodo ad ignorare la real natura (forma) dell'oggetto che gli viene specificato come parametro attuale;

12.6 Operatore `instanceof`

Visto che è lecito scambiarsi oggetti usando riferimenti ai loro antenati, potrebbe essere necessario sapere esattamente la forma dell'oggetto con cui si ha a che fare.

es

```
public class Impiegato extends Object // Ridondante
```

```
public class Quadro extends Impiegato
public class Segretario extends Impiegato
```

Se si riceve un oggetto usando un riferimento ad *Impiegato*, esso potrebbe essere anche realmente un *Quadro* o un *Segretario*

12.7 Casting

- Nel caso in cui si riceva un riferimento ad un oggetto e che (usando `instanceof`) si sappia che l'oggetto è realmente di una sottoclasse, si dovrebbe comunque usare quell'oggetto come se fosse della superclasse;
- La formalizzazione di polimorfismo rende invisibili tutte le specializzazioni proprie di quell'oggetto;
- Per redenere visibili tutte le caratteristiche dell'oggetto è necessario un *cast* esplicito;
- Se non ci fosse il cast, il metodo risulterebbe inaccessibile al compilatore;

Ogni tentativo di effettuare un cast di riferimenti ad oggetti è soggetto a regole precise.

Siano A,B,C 3 tipi:

- Gli upcasting sono sempre permessi, e infatti, non richiedono l'operatore di cast (vengono realizzati con una semplice assegnazione);
- Per i downcast il compilatore controlla che l'operazione sia almeno possibile $\rightarrow$ la classe del riferimento di destinazione deve essere una sottoclasse del riferimento di origine
  - Una volta che il compilatore abbia ammesso l'operazione, essa verrà controllata a runtime, per verificare che il tipo dell'oggetto riferito sia compatibile con il tipo di riferimento di destinazione;

12.8 Composizioni

- Implica una relazione a vita;
- Se il tutto è creato, anche le parti sono create;
- Quando il tutto muore, anche le parti muoiono;
- Il costruttore per il tutto deve costruire anche le parti.

12.9 Aggregazioni

- Nella aggregazione le parti sono create altrove
  - Esse sono inglobate nell'aggregante nel momento in cui esso viene costruito;
- Esiste un loro riferimento anche al di fuori dell'aggregante
  - in questo modo esse non cessano (forse) di esistere dopo la morte di esse

12.10 Associazioni

Nella semplice associazioni e vite degli oggetti, sia pure correlate, esistono in modo totalmente indipendente

12.11 Overloading e Overriding

Metodi con lo stesso nome, nella stessa classe, che svolgono lo stesso compito con diversi argomenti.

La lista degli argomenti deve essere diversa, perchè essa sola è usata come discriminante.

Il tipo di ritorno può essere diverso, ma non come discriminante per i metodi.

Per i costruttori il discorso è lo stesso dei metodi; il riferimento `this` può essere usato, nella prima linea di un costruttore, per invocarne un altro

Una sottoclasse può sovrappore un metodo ereditato e visibile, a patto che, rispetto al metodo della superclasse, il nuovo metodo abbia:

1. Lo stesso nome, tipo di ritorno e lista di argomenti;
2. Visibilità non inferiore

12.11.1 La parola chiave `super`

Viene utilizzata in una classe per riferirsi alla superclasse.

È usata per riferirsi ai membri della superclasse (attributi e metodi) con la dot notation.

I membri della superclasse accessibili con `super` sono ache quelli che la superclasse ha ereditato e non solo quelli esplicitamente definiti in essa.

Non si può usare `super.super.membro` per avvedere a membri di classi di livello superiore alla prima superclasse.

1. Invocazione di un `super-costruttore`

   Si può invocare uno dei costruttori della superclasse usando `super(...)` come prima linea nel proprio costruttore, con analoga sintassi di quella del proprio costruttore.

   Se come prima linea nel proprio costruttore non c'è nè `this` nè `super`, il compilatore pone una chiamata implicita a `super()`, cioè al costruttore di default della superclasse → in questo caso, se non esiste il costruttore di default nella superclasse, il compilatore genera un errore.

12.12 Costruzione ed inizializzazione di oggetti

- Viene allocata nello heap la memoria per il nuovo oggetto e sono inizializzate le variabili di istanza ai valori impliciti di default;

- Per ogni successiva chiamata ad un costruttore, cominciando dal costruttore individuato per primo, viene eseguita la sequenza di passi:

  1. Assegna i parametri del costruttore;
  2. Se è presente come prima riga, una chiamata esplicita a `this(...)`, richiama ricorsivamente il nuovo costruttore e poi passa al punto 5;
  3. Richiama l'implicito o esplicito `super(...)-costruttore`, eccetto per Object (non presenta superclassi);
  4. Esegue ogni inizializzazione esplicita delle variabili di istanza;
  5. Esegue il corpo del costruttore corrente.

12.13 La classe `Object`

È la radice di tutte le cassi. Una dichiarazione di classe senza la clausola `extends`, implicitamente usa *extends Object*.

Questo permette di sovrapporre numerosi metodi della classe *Object*.

12.13.1 Il metodo `equals`

- L'operatore `==` determina se 2 riferimenti sono identici (cioè se si riferiscono allo stesso, unico, oggetto);
- Il metodo `equals` determina se le due variabili si riferiscono ad oggetti (anche distinti) appartenenti alla stessa classe di equivalenza, rispetto ad una particolare relazione di equivalenza definita dall'utente;
- La realizzazione di equals nella classe *Object* fa uso di `==` $\rightarrow$ quindi le classi di equivalenza di `==` e di `equals` coincidono in *Object* (relazione di identità);
- Lo scopo del metodo è quello di essere sovrapposto nelle classi sviluppate dagli utenti, in modo da realizzare nuove relazioni di equivalenza;
- Alcune classi di sistema lo fanno già (String);
- Viene raccomandato di sovrappore, insieme con *equals*, anche il metodo *hashCode*.

12.13.2 Il metodo `toString`

Restituisce una rappresentazione `String` di un qualunque oggetto. Viene usato durante le conversioni automatiche a stringhe.

Occorer sovrappore questo metodo quando si vuole fornire una rappresentazione di un oggetto in forma (umanamente) leggibile.

I tipi primitivi sono rappresentati in forma di *String* usando il metodo statico `toString` della corrispondente classe *wrapper*

12.14 Le classi `wrapper`

I tipi primitivi non sono oggetti → Se si vogliono manipolare come oggetti, è possibile avvolgre un singolo valore con un opportuno oggetto, il cosiddetto *wrapper*:

Tipo primitivo	Classe wrapper
boolean	Boolean
byte	Byte
char	Character
short	Short
int	Integer
long	Long
float	Float
double	Double

12.14.1 Uso

Si può costruire un oggetto wrapper, passando il relativo valore al costruttore appropriato.

- Il valore da avvolgere può essere passato anche in una rappresentazione sotto forma di `String`;
- Il valore avvolto può essere estratto il metodo opportuno `...Value()`;
- Le classi wrapper sono utili quando si vogliono convertire i tipi primitivi.

12.15 Molteplicità

Si sfruttano gli arraylist.

13 Lezione del 26-04

13.1 Polimorfismo per inclusione (basato su upcast e downcast)

È uno strumento flessibile per realizzare strutture dati polimorfe.

Tuttavia il downcast necessario per estrarre elementi dalla struttura dati è verificabile solo a tempo di esecuzione (può generare errori a runtime che il compilatore non può identificare in anticipo).

13.2 Modificatori di accesso

Sono parole riservate che danno al compilatore informazioni sulla natura del codice, dei dati, delle classi. Un gruppo di modificatori possono essere usati, in combinazione con i precedenti, per qualificare quella caratteristica.

13.2.1 Generalità

Una caratteristica di una classe può essere:

- La classe stessa;
- Un attributo (variabile) della classe;
- Un metodo o un costruttore della classe.

Le sole variabili che possono essere controllate con i modificatori di accesso sono gli attributi (variabili di head) e non le variabili dei metodi (variabili di stack): una variabile di un metodo può essere vista solo dal metodo in cui si trova.

I metodi di accesso (mutualmente esclusivi) sono:

- public;
- protected;
- private.

Il solo modificatore di accesso permesso per una classe top-level è public: non esistono classi top-level protected o private.

Se non è presente il modificare, si intende "accesso consentito dallo stesso pacchetto in cui è situato quella caratteristica".

`public`

Modificatore più generoso. Accesso consentito a tutte le altre classi.

L'accesso ad un attributo o un metodo public è subordinato all'accesso alla classe che lo contiene

`private`

È il metodo meno generoso. Può essere usato solo per attributi o metodi, non per classi top-level

Le variabili private possono essere nascoste anche allo stesso oggetto che le possiede

```
class Complesso {
    private double reale , immag ;
```

```
}
class SubComplesso extends Complesso {

    SubComplesso(double r, double i){
real = r; // Illegale
    }
}
```

La classe SubComplesso eredita gli attributi della superclasse, ma quegli attributi possono essere usati solo dal codice della classe Complesso.

`Default`

Non è una parola riservata → si intende accesso di default quando non è presente nessun modificatore di accesso. Accesso consentito a tutte le classi nello stesso package.

`protected`

Accesso consentito a tutte le classi nello stesso package e a tutte le sottoclassi in pacchetti diversi, ma solo per ereditarietà: una sottoclasse in un altro pacchetto può accedere solo attraverso un riferimento ad un oggetto del proprio tipo (anche all'oggetto corrente, mediante `this`, o, esplicitamente, alla sua parte ereditata mediante `super`) o di un sottotipo.

13.2.2 Altri modificatori

`final`

Si applica a classi, metodi e variabili (non a costruttori).

Il significato varia da contesto a contesto, ma l'essenza è:

una caratteristica `final` non può essere modificata

Una classe final non può essere estesa, cioè non può avere sottoclassi.

Una variabile final è praticamente una costante, cioè può solo essere inizializzata.

NB: Un riferimento final ad un oggetto non può essere modificato (cioè riassegnato ad un altro oggetto), ma l'oggetto a cui esso fa riferimento si.

Un metodo final non può essere sovrapposto in una sottoclasse.

Non ha senso dire `final` ad un costruttore, perchè esso non è mai eridatato dalle sottoclassi, quindi mai sovrapponibile

`abstract`

Si applica solo a classi e a metodi. Un metodo abstract non possiede corpo (";" invece di "{...}").

Una classe può essere marcata abstract. In questo caso il compilatore suppone chessa contenga (anche per ereditarietà) metodi `abstract`: essa non potrà essere istanziata, anche se non contiene affatto metodi abstract.

Una classe deve essere marcata abstract se:

- Essa contiene almeno un metodo `abstract`, oppure;
- Essa eredita almeno un metodo `abstract`, per il quale non fornisce una realizzazione, oppure;
- Essa dichiara di implementare una interfaccia, ma non fornisce una realizzazione di tutti i metodi di quell'interfaccia.

In un certo senso `abstract` è opposto a `final` →

- Una classe final, per esempio non può essere specializzata;
- Una classe abstract, esiste solo per essere specializzata.

`static`

Si applica ad attributi, metodi ed anche a blocchi di codice che non fanno parte di metodi. In generale, una caratteristica `static` appartiene alla classe, non alle singole istanze → essa è unica, indipendentemente dal numero (anche 0) di istanze di quella classe.

Attributi static

L'inizializzazione di un attributo `static` avviene nel momento in cui la classe viene caricata in memoria (anche se non esisterà mai nessuna istanza di quella classe)

```
class Ecstatic{
    static int x = 0;
```

```
    Ecstatic () {
x++;
    }
}
```

L'accesso ad un attributo `static` di una classe può avvenire (con la dot-notation):

- O partendo da un riferimento ad una istanza di quella classe;
- O partendo dal nome stesso della classe.

```
System.out.println(Ecstatic.x);
Ecstatic e = new Ecstatic();
e.x = 100;
Ecstatic.x = 100;
```

Metodi static

Esistono nel momento in cui la classe viene caricata in memoria (anche senza istanze).

Non possono accedere a membri non static della stessa classe (perchè potrebbero non esistere). Un esempio è il metodo main

```
class Foo {
    static int i = 48;
    int j = 1
public static void main (String args[]){
i += 100;
// j *= 5 ; Per fortuna è commentata
    }
}
```

- Non possono essere sovrapposti da metodi `non-static`;
- Non possono sovrapporre metodi `non-static`.

Blocchi static

È lecito che una classe contenga blocchi di codice marcati `static`. Tali blocchi sono eseguiti una sola volta, nell'ordine in cui compaiono, quando la classe viene caricata in memoria ed, ovviamente, possono accedere (come i metodi `static`) solo a caratteristiche `static`

```
public class EsempioStatic{
    static double d = 1.23;
    static {
System.out.println("Codice static: d= " + d++);
    }

    public static void main (String args[]){
System.out.println("Main: d= " + d++);

    }

    static {
System.out.println("Codice static: d= " + d++);
    }
}
```

`native`

Si applica solo a metodi. Come per `abstract`, `native` indica che il corpo di un metodo deve essere trovato altrove, in questo caso all'esterno della JVM, in una libreria di codice dipendente dalla architettura fisica.

`transient`

Si applica solo agli attributi. Indica che quell'attributo contiene informazioni sensibili e che, nel caso in cui lo stato interno dell'oggetto debba essere slavato (nel gergo "serializzato"), il valore di quell'attributo non dovrà essere considerato.

`synchronized`

Si applica solo a metodi o a blocchi anonimi.

Controlla l'accesso al codice in programmi multi-thread.

`volatile`

Si applica solo agli attributi. Avverte il compilatore che quell'attributo può essere modificato in modo asincrono, in architettura multiprocessore.

13.2.3 Sommario

Modificatore	Classe	Attributo	Metodo	Costruttore	Blocco
public	✓	✓	✓	✓	
protected		✓	✓	✓	
"default"	✓	✓	✓	✓	✓
private		✓	✓	✓	
final	✓	✓	✓		
abstract	✓		✓		
static		✓	✓		✓
native			✓		
transient		✓			
volatile		✓			
synchronized			✓		✓

13.2.4 Singoletto

Uno dei problemi ricorrenti è il seguente → si vuole che una classe abbia una sola istanza. Se si scrivesse la classe nel classico modo, gli utilizzatoripotrebbero creare istanze a volontà.

Lo scopo della forma progettuale *Singleton* è quello di assicurare il progettista della claase che esisterà sempre una ed una sola instanza di quella classe